In [130…

```
# Initialize Otter
import otter
grader = otter.Notebook("hw5.ipynb")
```

CPSC 330 - Applied Machine Learning

Homework 5: Putting it all together

Associated lectures: All material till lecture 13

Instructions

rubric={points}

You will earn points for following these instructions and successfully submitting your work on Gradescope.

Before you start

- Please **read carefully Use of Generative AI policy** before starting the homework assignment.
- Review the CPSC 330 homework instructions for detailed guidance on completing and submitting assignments.

Group work instructions

You may work with a partner on this homework and submit your assignment as a group. Below are some instructions on working as a group.

- The maximum group size is 2.
- Use group work as an opportunity to collaborate and learn new things from each other.
- Be respectful to each other and make sure you understand all the concepts in the assignment well.
- It's your responsibility to make sure that the assignment is submitted by one of the group members before the deadline.
- You can find the instructions on how to do group submission on Gradescope here.
- If you would like to use late tokens for the homework, all group members must have the necessary late tokens available. Please note that the late tokens will be counted for all

members of the group.

Before submitting

- **Run all cells** (▶️ ▶️ button) to ensure the notebook executes cleanly from top to bottom.
  - Execution counts must start at **1** and be sequential.
  - Notebooks with missing outputs or errors may lose marks.
- **Do not upload or push data files** used in this lab to GitHub or Gradescope. (A `.gitignore` is provided to prevent this.)

Submitting on Gradescope

- Upload **only** your `.ipynb` file (with outputs shown) and any required output files. Do **not** submit extra files.
- If needed, refer to the Gradescope Student Guide.
- If your notebook is too large to render, also upload a **Web PDF** or **HTML** version.
  - You can create one using **File** → **Save and Export Notebook As**.
  - If you get an error when creating a PDF, try running the following commands in your lab directory:

    ```
    conda install -c conda-forge nbconvert-playwright
    jupyter nbconvert --to webpdf lab1.ipynb
    ```
  - Ensure all outputs are visible in your PDF or HTML file; TAs cannot grade your work if outputs are missing.

Note: Unlike previous assignments, this one is open-ended and project-style. Treat it as an opportunity to explore, experiment, and learn.

Imports

Imports

Points: 0

In [131…

```
# TEMP IMPORTS
from hashlib import sha1

import matplotlib.pyplot as plt
import numpy as np
```

```
import pandas as pd

plt.rcParams["font.size"] = 16

from sklearn.dummy import DummyClassifier
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import (
    GridSearchCV,
    RandomizedSearchCV,
    cross_val_score,
    cross_validate,
    train_test_split,
)
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OrdinalEncoder
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import make_column_transformer
from sklearn.feature_selection import RFE
from sklearn.feature_selection import RFECV
from sklearn.metrics import precision_score, recall_score, f1_score
```

Introduction

In this homework you will be working on an open-ended mini-project, where you will put all the different things you have learned so far together to solve an interesting problem.

A few notes and tips when you work on this mini-project:

Tips

1. This mini-project is open-ended, and while working on it, there might be some situations where you'll have to use your own judgment and make your own decisions (as you would be doing when you work as a data scientist). Make sure you explain your decisions whenever necessary.
2. **Do not include everything you ever tried in your submission** -- it's fine just to have your final code. That said, your code should be reproducible and well-documented. For example, if you chose your hyperparameters based on some hyperparameter optimization experiment, you should leave in the code for that experiment so that someone else could re-run it and obtain the same hyperparameters, rather than mysteriously just setting the hyperparameters to some (carefully chosen) values in your code.
3. If you realize that you are repeating a lot of code try to organize it in functions. Clear presentation of your code, experiments, and results is the key to be successful in this

lab. You may use code from lecture notes or previous lab solutions with appropriate attributions.

Assessment

We plan to grade fairly and leniently. We don't have some secret target score that you need to achieve to get a good grade. **You'll be assessed on demonstration of mastery of course topics, clear presentation, and the quality of your analysis and results.** For example, if you just have a bunch of code and no text or figures, that's not good. If you do a bunch of sane things and get a lower accuracy than your friend, don't sweat it.

A final note

Finally, this style of this "project" question is different from other assignments. It'll be up to you to decide when you're "done" -- in fact, this is one of the hardest parts of real projects. But please don't spend WAY too much time on this... perhaps "a few hours" (10-14 hours???) is a good guideline for this project . Of course if you're having fun you're welcome to spend as much time as you want! But, if so, try not to do it out of perfectionism or getting the best possible grade. Do it because you're learning and enjoying it. Students from the past cohorts have found such kind of labs useful and fun and I hope you enjoy it as well.

1. Pick your problem and explain the prediction problem

---

rubric={points:3}
In this mini project, you have the option to choose on which dataset you will be working on. The tasks you will need to carry on will be similar, independently of your choice.

Option 1

You can choose to work on a classification problem of predicting whether a credit card client will default or not. For this problem, you will use Default of Credit Card Clients Dataset. In this data set, there are 30,000 examples and 24 features, and the goal is to estimate whether a person will default (fail to pay) their credit card bills; this column is labeled "default.payment.next.month" in the data. The rest of the columns can be used as features. You may take some ideas and compare your results with the associated research paper, which is available through the UBC library.

Option 2

You can choose to work on a regression problem using a dataset of New York City Airbnb listings from 2019. As usual, you'll need to start by downloading the dataset, then you will try to predict `reviews_per_month`, as a proxy for the popularity of the listing. Airbnb could use this sort of model to predict how popular future listings might be before they are posted, perhaps to help guide hosts create more appealing listings. In reality they might instead use something like vacancy rate or average rating as their target, but we do not have that available here.

> Note there is an updated version of this dataset with more features available here. The features were are using in `listings.csv.gz` for the New York city datasets. You will also see some other files like `reviews.csv.gz`. For your own interest you may want to explore the expanded dataset and try your analysis there. However, please submit your results on the dataset obtained from Kaggle.

Your tasks:

1. Review the available datasets and choose the one you find most interesting. It may help to read through the dataset documentation on Kaggle before deciding.
2. Once you've selected a dataset, take time to understand the problem it represents and the meaning of each feature. Use the Kaggle documentation to guide you.
3. Download the dataset and load it into a pandas DataFrame.
4. Write a few sentences summarizing your initial thoughts about the problem and the dataset.

Solution_1

Points: 3

We chose option 1, the classification problem about whether a credit card client will default or not.

We chose this option because we were curious about how a machine learning model would be able to use the combination of different features given to identify something as ambiguous as whether someone would pay their credit card on time. When reading the associated research paper, it was interesting seeing real-world applications of the models we've been covering in class (like KNN, logistic regression, and classification trees), as well as how the resulting data was actually interpreted by researchers. We were also curious how our current implementation of SKLearn's models would perform compared to the (assumedly more advanced) ones used in the research paper and whether we would be able to replicate their results.

In [132…

```
credit_df = pd.read_csv("data/UCI_Credit_Card.csv", index_col=0)
credit_df.shape
credit_df.head()
```

Out[132…

	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	PAY_5
ID										
1	20000.0	2	2	1	24	2	2	-1	-1	-2
2	120000.0	2	2	2	26	-1	2	0	0	0
3	90000.0	2	2	2	34	0	0	0	0	0
4	50000.0	2	2	1	37	0	0	0	0	0
5	50000.0	1	2	1	57	-1	0	-1	0	0

5 rows × 24 columns

2. Data splitting

rubric={points:2}

Your tasks:

1. Split the data into train (70%) and test (30%) portions with `random_state=123`.

> If your computer cannot handle training on 70% training data, make the test split bigger.

Solution_2

Points: 2

In [133…

```
X = credit_df.drop(columns=["default.payment.next.month"], axis=1)
y = credit_df["default.payment.next.month"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_sta
X_train.shape
```

Out[133…

```
(21000, 23)
```

3. EDA

rubric={points:10}

Your tasks:

1. **Perform exploratory data analysis (EDA)**: Conduct an initial exploration of the training set to better understand its characteristics.
2. **Summarize and visualize the data**: Include at least **two summary statistics** and **two visualizations** that you find informative. For each, write **one sentence** explaining what insight it provides.
3. **Record your observations**: Summarize your **initial observations** about the dataset based on your EDA.
4. **Select evaluation metrics**: Choose one or more **appropriate metrics** for assessing model performance and briefly justify your choice.

Solution_3

Points: 10

In [134…

```
credit_df.isnull().values.any()
print(credit_df["EDUCATION"].value_counts()) # 51
print(credit_df["MARRIAGE"].value_counts()) # 323 for 3, 54 for 0
print(credit_df["PAY_0"].value_counts())
print(credit_df["PAY_2"].value_counts())
print(credit_df["PAY_3"].value_counts())
print(credit_df["PAY_4"].value_counts())
print(credit_df["PAY_5"].value_counts())
print(credit_df["PAY_6"].value_counts())

num_defaulted = sum(credit_df["default.payment.next.month"])
print(num_defaulted) # 6636

print(credit_df["SEX"].value_counts())
```

```
EDUCATION
2    14030
1    10585
3     4917
5      280
4      123
6       51
0       14
Name: count, dtype: int64
MARRIAGE
2    15964
1    13659
3      323
0       54
Name: count, dtype: int64
PAY_0
 0    14737
-1     5686
 1     3688
-2     2759
 2     2667
 3      322
 4       76
 5       26
 8       19
 6       11
 7        9
Name: count, dtype: int64
PAY_2
 0    15730
-1     6050
 2     3927
-2     3782
 3      326
 4       99
 1       28
 5       25
 7       20
 6       12
 8        1
Name: count, dtype: int64
PAY_3
 0    15764
-1     5938
-2     4085
 2     3819
 3      240
 4       76
 7       27
 6       23
 5       21
 1        4
 8        3
Name: count, dtype: int64
PAY_4
 0    16455
```

```
-1     5687
-2     4348
 2     3159
 3      180
 4       69
 7       58
 5       35
 6        5
 1        2
 8        2
Name: count, dtype: int64
PAY_5
 0    16947
-1     5539
-2     4546
 2     2626
 3      178
 4       84
 7       58
 5       17
 6        4
 8        1
Name: count, dtype: int64
PAY_6
 0    16286
-1     5740
-2     4895
 2     2766
 3      184
 4       49
 7       46
 6       19
 5       13
 8        2
Name: count, dtype: int64
6636
SEX
2    18112
1    11888
Name: count, dtype: int64
```

EDA:

First, we can analyze whether a column has any NaN values. Using isnull(), we see that no values inside the table are NaN. However, looking at the descriptions of each column on Kaggle, we can see that there are various values for unknown values, such as 6 for unknown EDUCATION, 3 and 0 for unspecified MARRIAGE status, and 9 for 9 months or more of PAY delay.

EDUCATION has 51 unknowns, MARRIAGE 377, all PAY 's have 0 values that are 9, so the impact of possible unknowns is fortunately relatively small when we consider that the total dataset has 30,000 entries.

We can also learn that out of the 30,000 entries, only 6636 entries resulted in the credit card being defaulted, meaning the data is quite imbalanced, so accuracy alone is probably not be the best indicator for determining a model's accuracy. Since this is a classification problem, we should instead evaluate the models using their precision, recall, and F1 scores alongside model accuracy.

Summary Statistics: (Observed from the graphs shown on Kaggle)

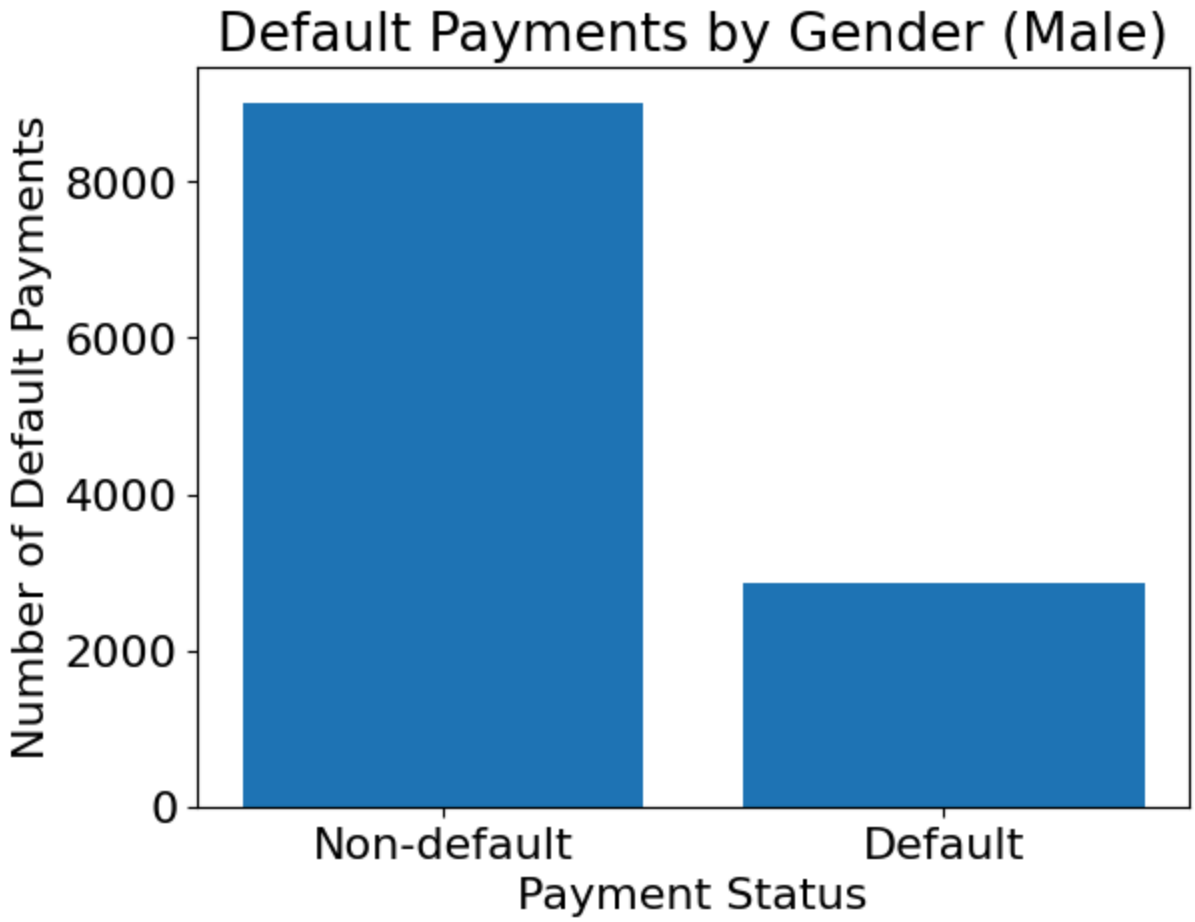
1. 6636/30000 cases are default cases, meaning our dataset is imbalanced, meaning even a baseline model (like DummyClassifier) would only have an error of 22.12%.
2. The ratio between men and women in the study is skewed towards women, with around 7000 more entries.
3. Similarly, the age of people surveyed is heavily skewed towards people on the younger side, with the most common age range being 26.80-29.70. The imabalance could possibly indicate that credit card use is more common in younger people.
4. The mean for all PAY columns was negative, meaning the number of people paying their card on time (a value of -1) is enough to offset all of the people who have late payments of up to 9 or more months.

In [135…

```
m_non = len(credit_df[(credit_df['SEX'] == 1) & (credit_df['default.payment.next.mo
m_def = len(credit_df[(credit_df['SEX'] == 1) & (credit_df['default.payment.next.mo
def_counts = [m_non, m_def]
labels = ['Non-default', 'Default']

plt.bar(labels, def_counts)
plt.title("Default Payments by Gender (Male)")
plt.xlabel("Payment Status")
plt.ylabel("Number of Default Payments")
```

Out[135…

```
Text(0, 0.5, 'Number of Default Payments')
```

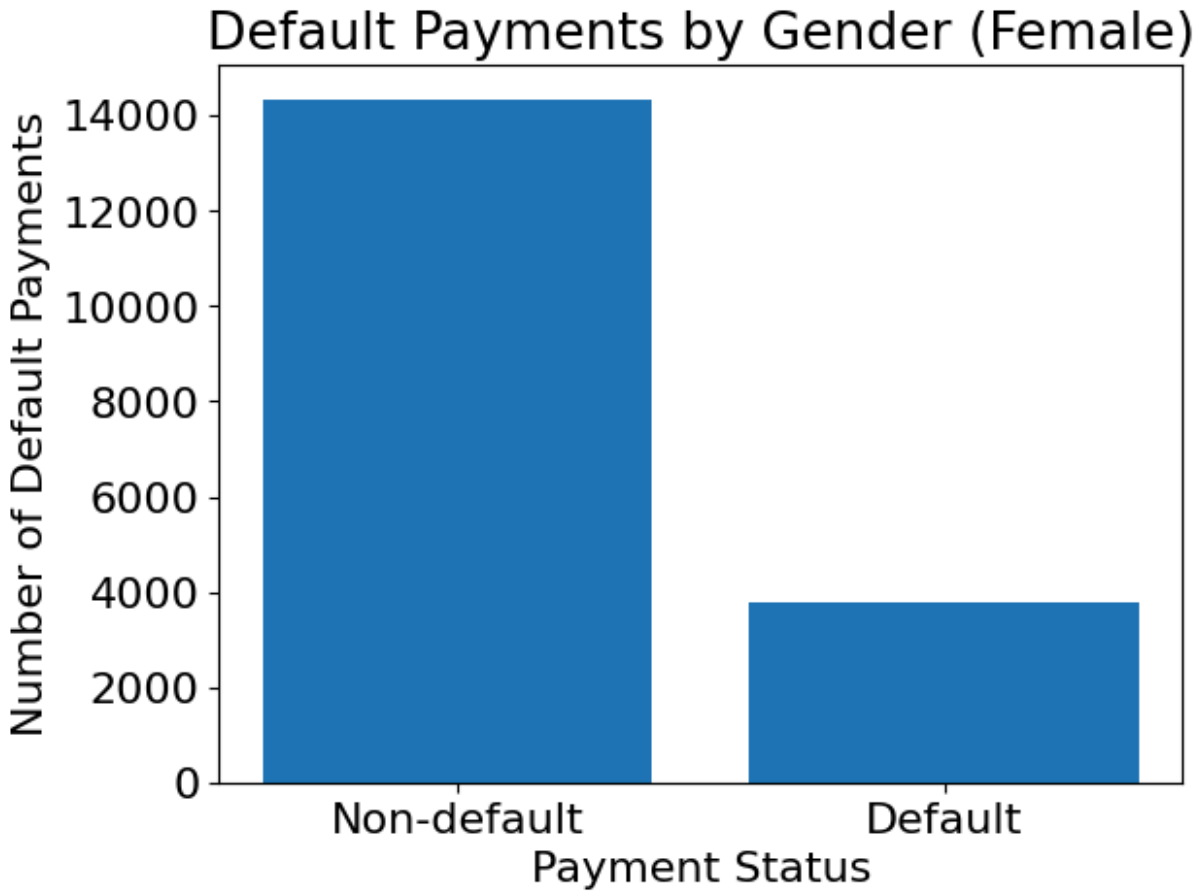


In [136…

```
f_non = len(credit_df[(credit_df['SEX'] == 2) & (credit_df['default.payment.next.mo
f_def = len(credit_df[(credit_df['SEX'] == 2) & (credit_df['default.payment.next.mo
non_counts = [f_non, f_def]
labels = ['Non-default', 'Default']
plt.bar(labels, non_counts)
plt.title("Default Payments by Gender (Female)")
plt.xlabel("Payment Status")
plt.ylabel("Number of Default Payments")
```

Out[136…

```
Text(0, 0.5, 'Number of Default Payments')
```



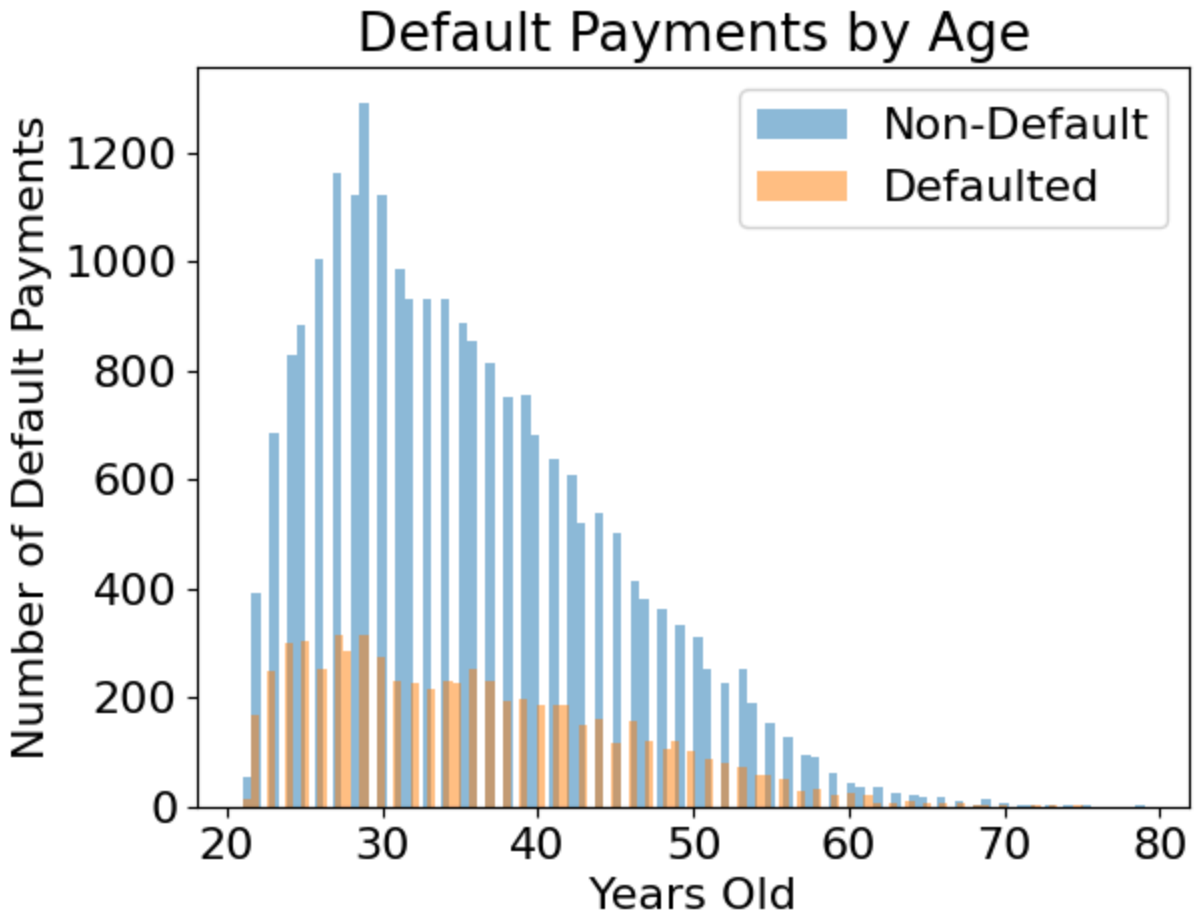
Since the ratio between non-default payments and default payments is relatively similar for both genders, it indicates that there's not a strong correlation between gender and the likeliness of the person defaulting a payment alone. We'll keep gender as a feature for now because it's possible that gender in combination with the other features may have a big impact on the predictions of the model.

In [137…

```
age_non = credit_df[credit_df['default.payment.next.month']==0]
age_def=credit_df[credit_df['default.payment.next.month']==1]

plt.hist(age_non['AGE'], bins=100, alpha=0.5, label='Non-Default')
plt.hist(age_def['AGE'], bins=100, alpha=0.5, label='Defaulted')
plt.title("Default Payments by Age")
plt.xlabel("Years Old")
plt.ylabel("Number of Default Payments")
plt.legend(loc='upper right')
```

Out[137…

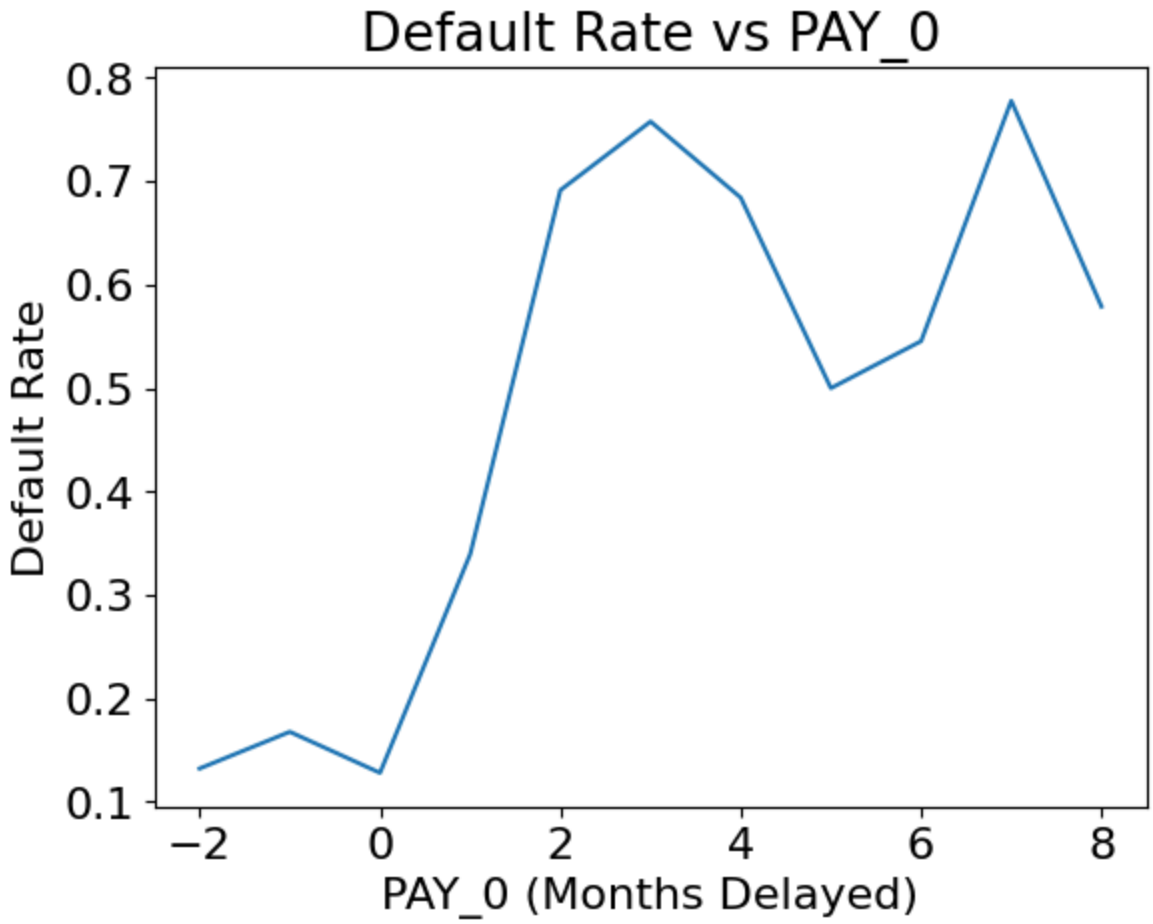
```
<matplotlib.legend.Legend at 0x1973a835bd0>
```



Similarly, when we look at the age distribution of those that paid on time and compare it to those that defaulted payments, their distributions are very similar. Both show a trend towards a higher number of younger credit card users, with a much less dramatic peak in those that defaulted payments, possibly indicating that someone who defaults a payment may do it regardless of age. In fact, the ratio of older credit card users that default payments compared to older credit card users that pay on time is significantly higher than younger credit card users.

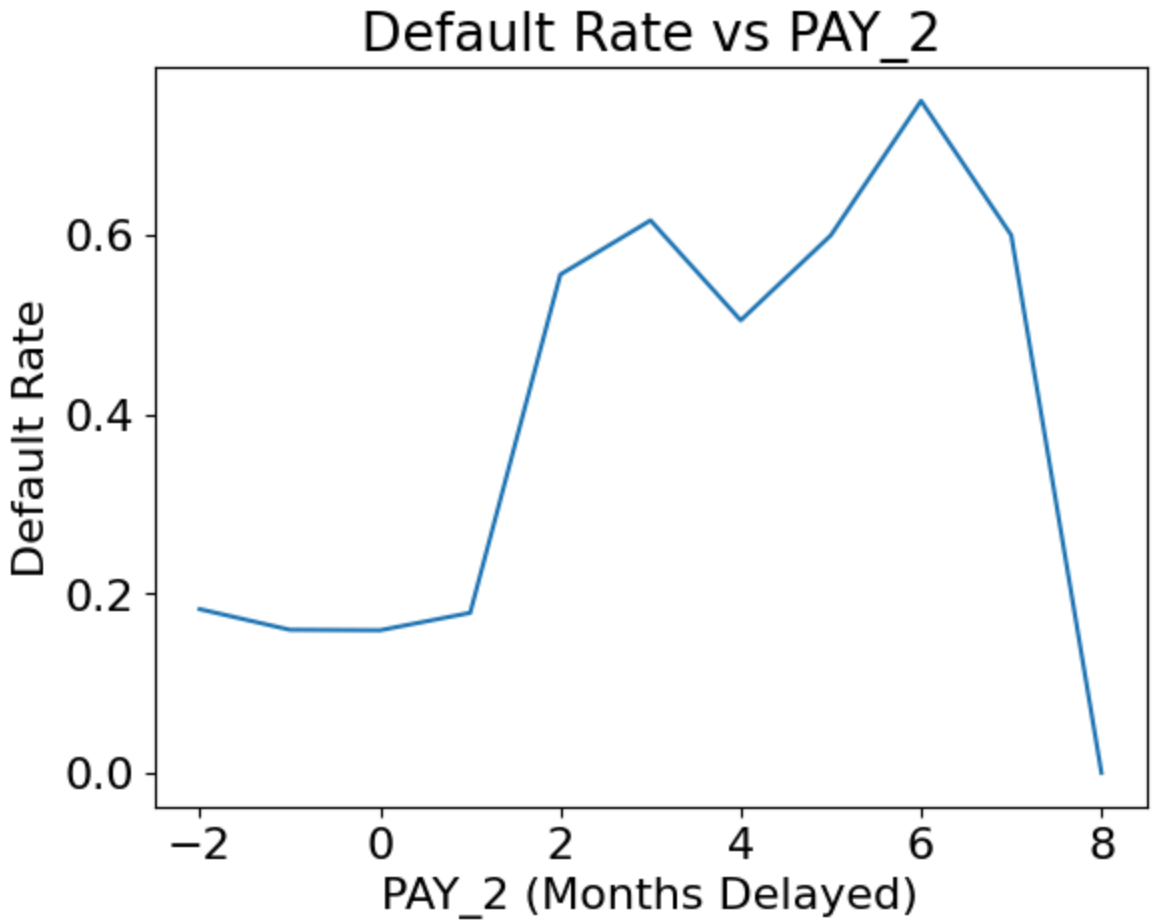
In [138…

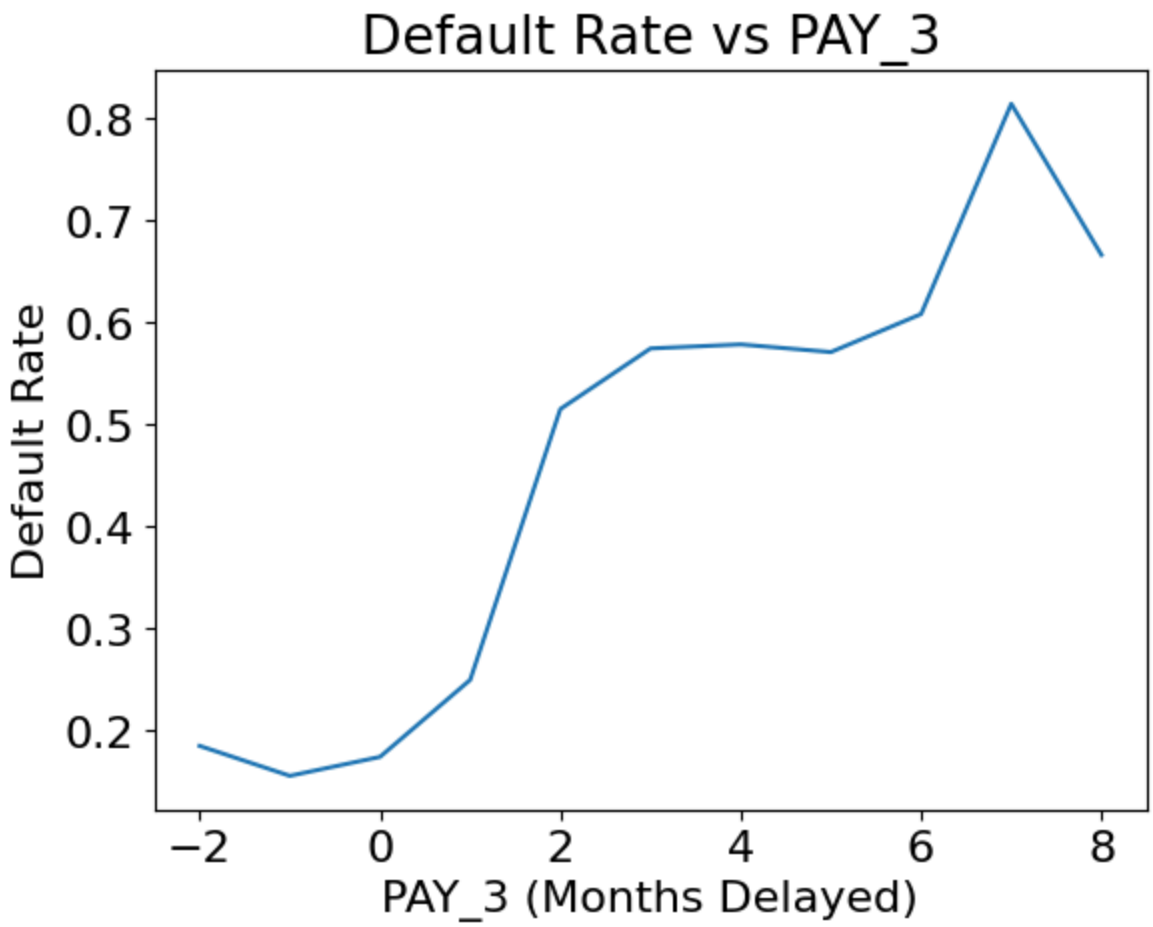
```
default_rate = credit_df.groupby("PAY_0")["default.payment.next.month"].mean()

plt.plot(default_rate.index, default_rate.values)
plt.xlabel("PAY_0 (Months Delayed)")
plt.ylabel("Default Rate")
plt.title("Default Rate vs PAY_0")
plt.show()
```

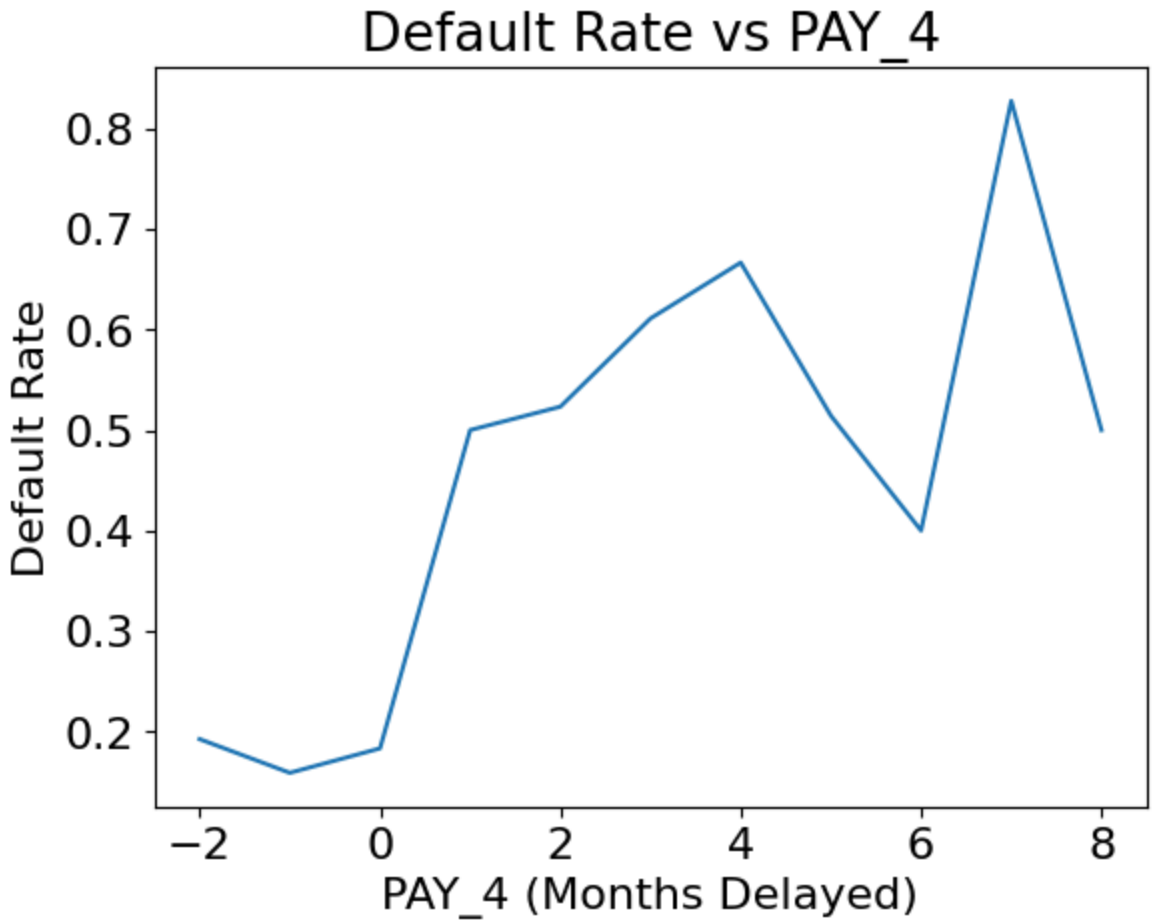


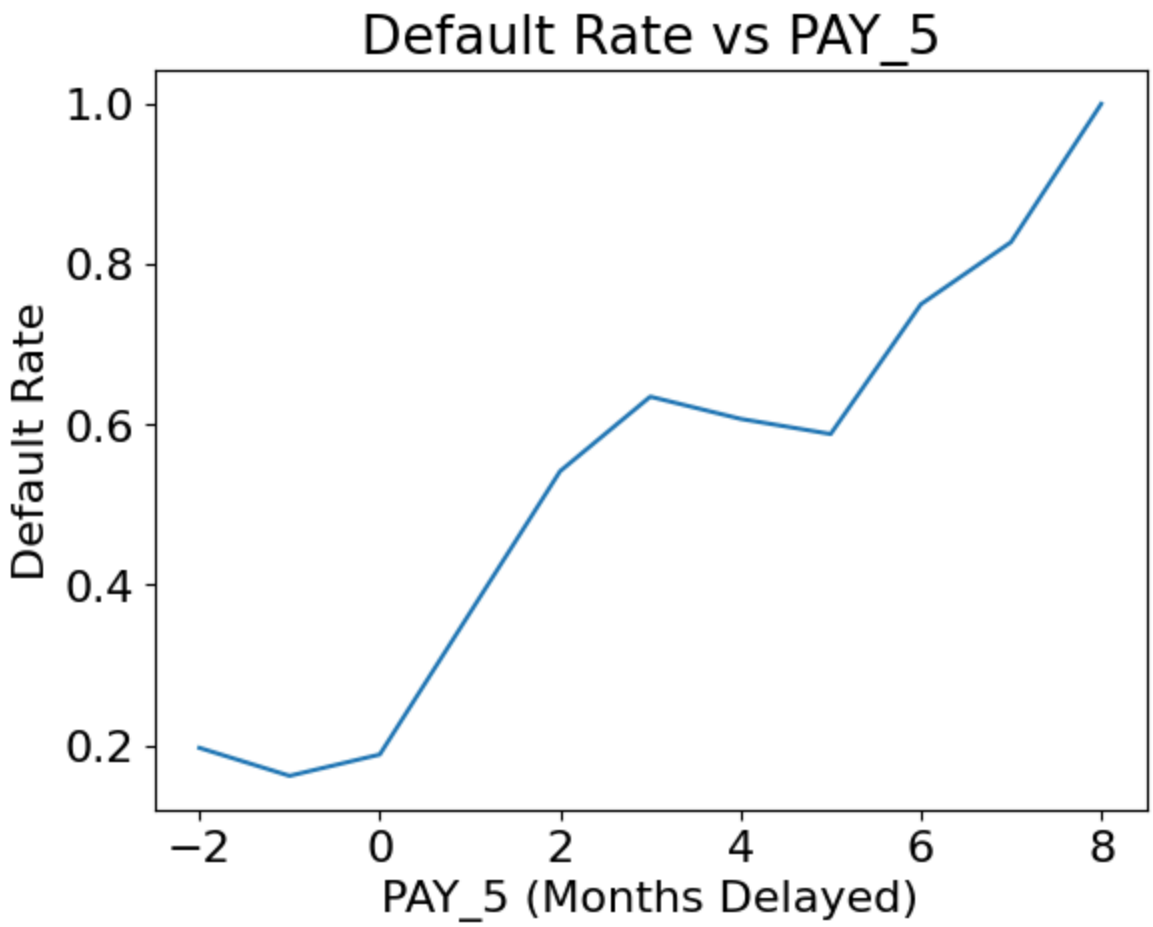
This graph of Default Rate vs PAY_0 shows that the default rate and pay status is heavily correlated with each other. Below, we see that this trend follows for the other PAY_ columns.

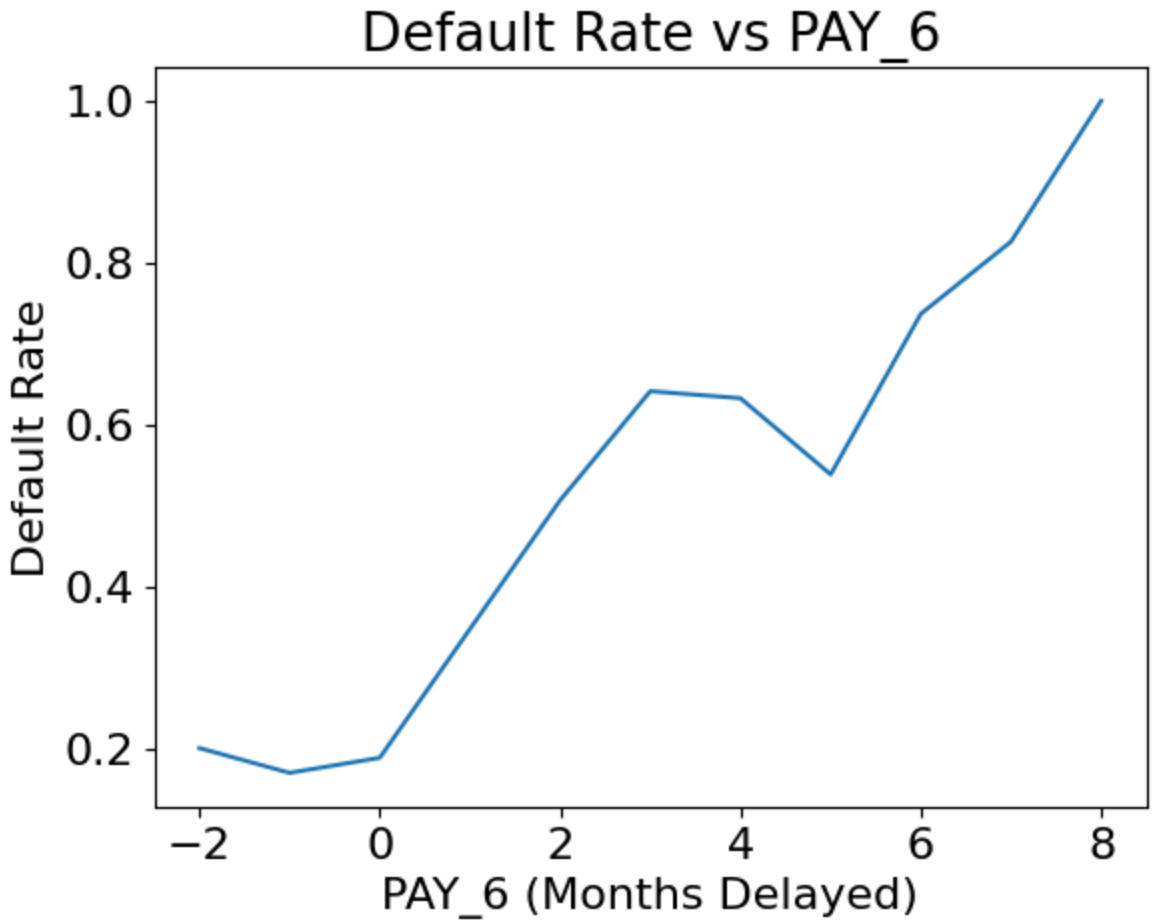
In [12]:
```
for i in range(2, 7):
    default_rate = credit_df.groupby(f"PAY_{i}")["default.payment.next.month"].mean
    plt.plot(default_rate.index, default_rate.values)
    plt.xlabel(f"PAY_{i} (Months Delayed)")
    plt.ylabel("Default Rate")
    plt.title(f"Default Rate vs PAY_{i}")
    plt.show()
```


Default Rate vs PAY_2
Default Rate
0.0
0.2
0.4
0.6
−2
0
2
4
6
8
PAY_2 (Months Delayed)


Default Rate vs PAY_3
Default Rate
0.2
0.3
0.4
0.5
0.6
0.7
0.8
−2
0
2
4
6
8
PAY_3 (Months Delayed)


Default Rate vs PAY_4
Default Rate
0.8
0.7
0.6
0.5
0.4
0.3
0.2
−2
0
2
4
6
8
PAY_4 (Months Delayed)


Default Rate vs PAY_5
Default Rate
1.0
0.8
0.6
0.4
0.2
−2
0
2
4
6
8
PAY_5 (Months Delayed)



Initial Observations:

Overall, it's very difficult to interpret all of the columns alone, as seen by the similarity in graphs and previous analysis, with the exception of the PAY_X columns. These columns are shown to be heavily correlated with the default rate. The data also has quite a lot of skewing, both in some categories (age, gender) as well as the category we'll be trying to predict (default.payment.next.month). These factors might make it more difficult to interpret the data correctly, especially for machine models which are affected a lot by skewed data. For this reason, we'll likely have to do a lot of preprocessing on the data later.

Fortunately, the data itself doesn't have any NaN values or many unknown values. The data can also be easily categorized into binary, numeric, categorical, and ordinal, which will make column processing much easier.

Evaluation Metrics:

Since the data is very imbalanced, we can't base model efficiency off accuracy alone. As this is a classification problem, we can use precision, recall, and F1-scores to judge models. Between recall and precision, if a model is unable to have both (low F1 score), recall is likely the more important score to consider as banks would likely prioritize detecting a higher amount of default risks (banks can't risk missing a true positive) rather than focusing on the precision of the default detections (false positives are not as risky for banks).

4. Feature engineering

rubric={points:1}

Your tasks:

1. **Perform feature engineering**: Create new features that are relevant to the problem and use this updated feature set in the following exercises. You may need to iterate between **feature engineering** and **preprocessing** to refine your features and improve model performance.

Solution_4

Points: 1

In [139…

```
for i in range(1, 7):
    bill_col = f"BILL_AMT{i}"
    pay_col = f"PAY_AMT{i}"
    new_col = f"BILL_PAY{i}"
    X_train[new_col] = X_train[pay_col] / (X_train[bill_col] + 1e-5)
    X_test[new_col] = X_test[pay_col] / (X_test[bill_col] + 1e-5)

pay_cols = [f"PAY_{i}" for i in [0,2,3,4,5,6]]

X_train["MAX_DELAY"] = X_train[pay_cols].max(axis=1)
X_test["MAX_DELAY"] = X_test[pay_cols].max(axis=1)
X_train["NUM_DELAY"] = (X_train[pay_cols] > 0).sum(axis=1)
X_test["NUM_DELAY"] = (X_test[pay_cols] > 0).sum(axis=1)
X_train["AVG_DELAY"] = X_train[pay_cols].mean(axis=1)
X_test["AVG_DELAY"] = X_test[pay_cols].mean(axis=1)
```

It is hard to draw information from just payment amount, so we decided to combine bill amount and payment amount into columns that consists of the ratio between payment:bill

amount.

As seen above, the PAY_X columns seem to have a strong correlation with the default rate. This made us inclined to extract useful generalizations from the 6 columns.

5. Preprocessing and transformations

rubric={points:10}

Your tasks:

1. **Identify feature types**: Determine the different types of features in your dataset (e.g., numerical, categorical, ordinal, text) and specify the transformations you would apply to each type.
2. **Define a column transformer (if needed)**: Implement a `ColumnTransformer` to apply the appropriate preprocessing steps to each feature type.

Solution_5

Points: 10

Before making a preprocessor, we want to clean up the data to hopefully boost the performance of the models.

In [140…

```
for df in [X_train, X_test]:
    df["EDUCATION"] = df["EDUCATION"].replace({0: 4, 5: 4, 6: 4}).astype(str)
```

We decided to make an adjustment to the EDUCATION column. We changes all 5's and 6's into 4's. Functionally, 4, 5, and 6 have a very similar purpose, meaning that combining these categories will have little to no effect on performance.

In [141…

```
numeric_features = [
    "LIMIT_BAL", "AGE",
    "PAY_0", "PAY_2", "PAY_3", "PAY_4", "PAY_5", "PAY_6",
    "BILL_AMT1", "BILL_AMT2", "BILL_AMT3",
    "BILL_AMT4", "BILL_AMT5", "BILL_AMT6",
    "PAY_AMT1", "PAY_AMT2", "PAY_AMT3",
    "PAY_AMT4", "PAY_AMT5", "PAY_AMT6",
    "BILL_PAY1","BILL_PAY2", "BILL_PAY3",
    "BILL_PAY4","BILL_PAY5", "BILL_PAY6",
    "MAX_DELAY", "NUM_DELAY", "AVG_DELAY"
```

```
]

ordinal_features = ["EDUCATION"]

binary_features = ["SEX"]

categorical_features = ["MARRIAGE"]
```

In [142…

```
numeric_transformer = make_pipeline(
    SimpleImputer(strategy="median"),
    StandardScaler()
)

education_levels = [['4', '3', '2', '1']]

ordinal_transformer = OrdinalEncoder(categories=education_levels)

binary_transformer = OneHotEncoder(drop="if_binary", dtype=int)

categorical_transformer = make_pipeline(
    SimpleImputer(strategy="most_frequent"),
    OneHotEncoder(handle_unknown="ignore", sparse_output=False)
)
```

In [143…

```
preprocessor = make_column_transformer(
    (numeric_transformer, numeric_features),
    (ordinal_transformer, ordinal_features),
    (binary_transformer, binary_features),
    (categorical_transformer, categorical_features),
)
```

In [144…

```
credit_df.head()
X_train_pp = preprocessor.fit(X_train)
column_names = preprocessor.get_feature_names_out()
X_train_pp = pd.DataFrame(preprocessor.transform(X_train),
                          columns = column_names,
                          index=X_train.index
                        )
X_train_pp
```

Out[144…

ID	pipeline-1__LIMIT_BAL	pipeline-1__AGE	pipeline-1__PAY_0	pipeline-1__PAY_2	pipeline-1__PAY_3	pipeline-1__PAY_4	pipeline-1__PAY_5	pipe 1__P
16396	1.168355	0.054187	0.013770	0.114774	0.143483	0.192754	0.232531	0.25
21449	2.090017	-0.597108	-0.878738	-0.722412	-0.692571	0.192754	0.232531	0.25
20035	-0.060527	0.922579	-1.771246	-1.559598	-1.528626	-1.518801	-1.526210	-1.48
25756	-0.367748	-0.597108	0.013770	0.114774	0.143483	0.192754	0.232531	0.25
1439	-0.905384	2.008070	0.906278	1.789147	0.143483	0.192754	0.232531	0.25
...	...	...	...	...	...	...	...	
28637	1.629186	0.162736	0.013770	0.114774	0.143483	0.192754	0.232531	1.99
17731	1.475576	2.008070	0.906278	-1.559598	-1.528626	-0.663024	0.232531	-0.61
28031	-0.905384	-0.705657	0.013770	0.114774	0.143483	0.192754	0.232531	0.25
15726	-1.058995	-1.574049	0.013770	0.114774	0.143483	0.192754	0.232531	0.25
19967	1.552381	0.054187	-1.771246	-1.559598	-1.528626	-1.518801	-1.526210	-1.48

21000 rows × 35 columns

6. Baseline model

rubric={points:2}

Your tasks:

1. **Establish a baseline**: Use one of `scikit-learn`'s baseline models (e.g., `DummyClassifier` or `DummyRegressor`, depending on your task) and report the results. This will serve as a reference point for evaluating the performance of your more advanced models.

Solution_6

Points: 2

In [145…

```
dummy = DummyClassifier()
```

In [146…
```
dummy.fit(X_train, y_train)
```

Out[146…
```
▾ DummyClassifier
▸ Parameters
```

In [147…
```
dummy.score(X_train, y_train)
```

Out[147…
```
0.7767619047619048
```

7. Linear models

rubric={points:10}

Your tasks:

1. **Train a linear model**: Use a linear model as your first real attempt at solving the problem.
2. **Tune hyperparameters**: Perform hyperparameter tuning to explore different values of the model's complexity parameter.
3. **Evaluate with cross-validation**: Report the cross-validation scores along with their standard deviation.
4. **Summarize findings**: Summarize your results, highlighting key observations from your experiments.

Solution_7

Points: 10

Since this is a classification problem, we'll be using a Logistic Regression Model. Overall, it didn't have the worst accuracy due to the data imabalance, but it could definitely be improved by hyperparameter tuning. Right now it's only performing about 3% better than the Dummy Classifier.

In [148…
```
lr = LogisticRegression(random_state=123) # using default hyperparameters
pipe_lr = make_pipeline(preprocessor, lr)
scores = pd.DataFrame(cross_validate(pipe_lr, X_train, y_train, return_train_score=
scores.mean()
```

Out[148…

```
fit_time       0.094311
score_time     0.006770
test_score     0.809095
train_score    0.809702
dtype: float64
```

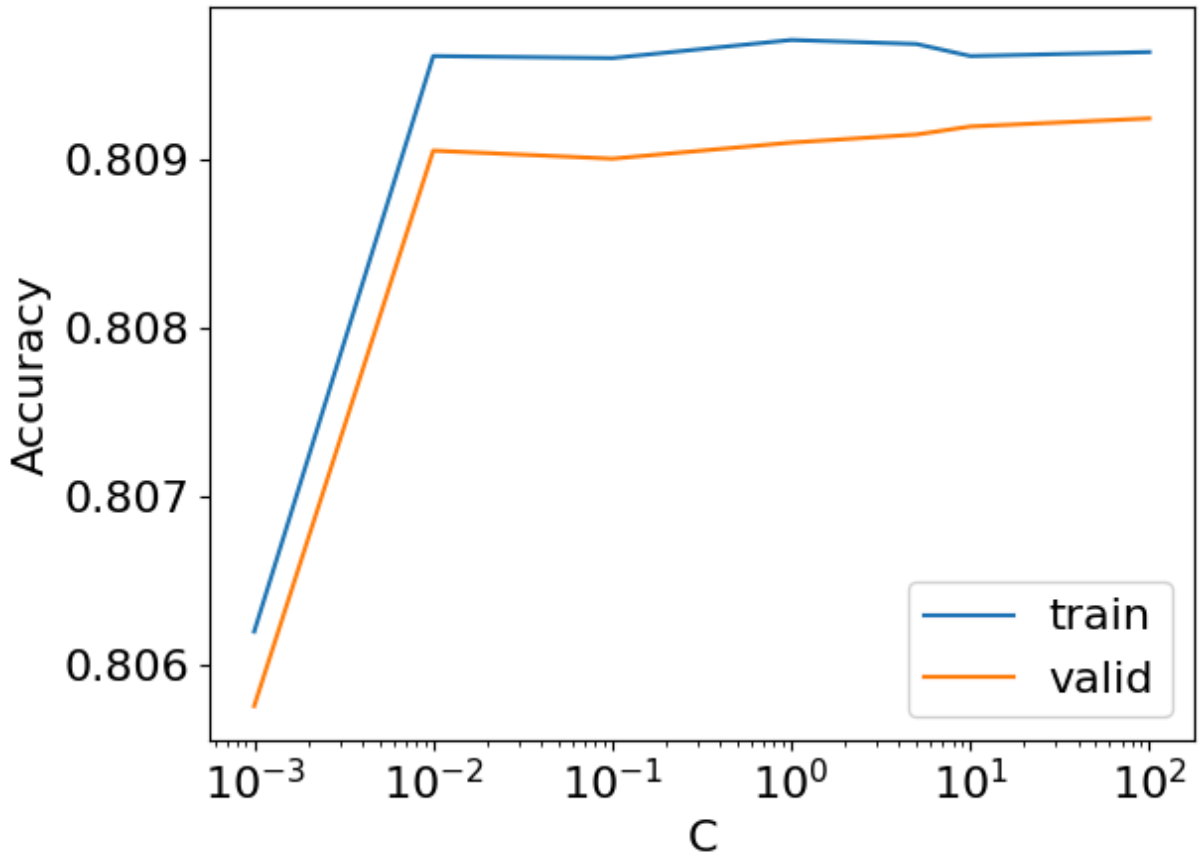
In [149…

```
C_vals = [0.001, 0.01, 0.1, 1, 5, 10, 100]
Max_iter = [1, 10, 100, 10000]
pipe_lr = make_pipeline(preprocessor, LogisticRegression(random_state=123))
param_grid = {
    "logisticregression__C": C_vals,
    "logisticregression__max_iter" : Max_iter
}

grid_search = GridSearchCV(pipe_lr, param_grid=param_grid, n_jobs=-1)
grid_search.fit(X_train, y_train)
best_C =  grid_search.best_params_.get("logisticregression__C")
best_Max_iter = grid_search.best_params_.get("logisticregression__max_iter")
best_score = grid_search.best_score_
print(best_C)
print(best_score)
print(best_Max_iter)
```

```
100
0.8092380952380953
100
```

In [150…

```
# taken from hw4
train_scores = []
cv_scores = []
std_scores = []
fit_times = []
score_times = []

for C in C_vals:
    #     print(C)
    pipe = make_pipeline(
        preprocessor,
        LogisticRegression(max_iter=100, C=C, random_state=123),
    )
    cv_results = cross_validate(
        pipe, X_train, y_train,
        return_train_score=True
    )

    train_scores.append(cv_results["train_score"].mean())
    cv_scores.append(cv_results["test_score"].mean())
    std_scores.append(np.std(cv_scores))
    fit_times.append(cv_results["fit_time"].mean())
    score_times.append(cv_results["score_time"].mean())

plt.semilogx(C_vals, train_scores, label="train")
plt.semilogx(C_vals, cv_scores, label="valid")
plt.legend()
plt.xlabel("C")
plt.ylabel("Accuracy");
```



In [151…

```
pd.DataFrame({"C": C_vals, "train": train_scores, "cv": cv_scores, "std": std_score
```

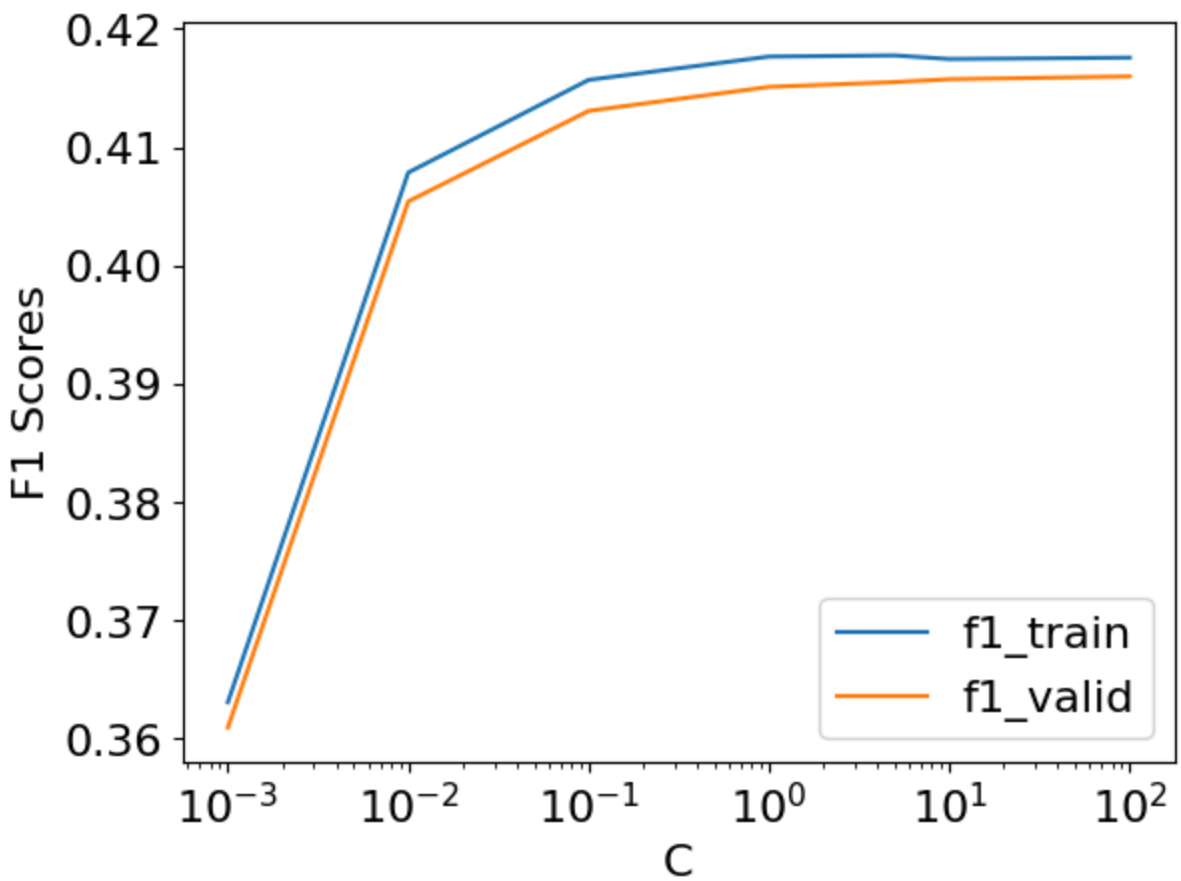
Out[151…

	C	train	cv	std	fit_time	score_time
0	0.001	0.806202	0.805762	0.000000	0.058994	0.006334
1	0.010	0.809607	0.809048	0.001643	0.065542	0.006374
2	0.100	0.809595	0.809000	0.001538	0.091681	0.005936
3	1.000	0.809702	0.809095	0.001423	0.089233	0.006364
4	5.000	0.809679	0.809143	0.001325	0.090662	0.006461
5	10.000	0.809607	0.809190	0.001244	0.113605	0.007267
6	100.000	0.809631	0.809238	0.001177	0.097232	0.006792

In [152…

```
# taken from hw4
f1_train_scores = []
f1_cv_scores = []
f1_std_scores = []
prec_train_scores = []
prec_cv_scores = []
rec_train_scores = []
rec_cv_scores = []

for C in C_vals:
    #     print(C)
```

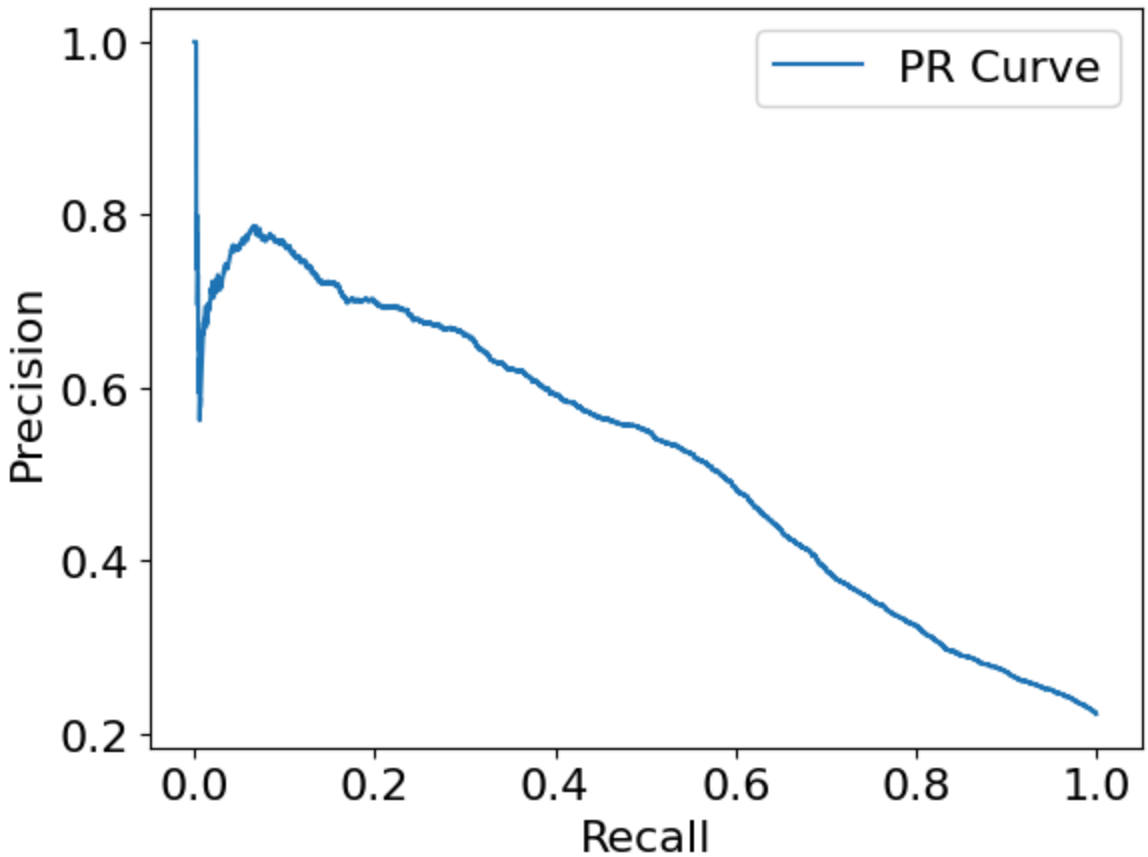
```
    pipe = make_pipeline(
        preprocessor,
        LogisticRegression(max_iter=100, C=C, random_state=123),
    )
    cv_results = cross_validate(
        pipe, X_train, y_train,
        scoring=["f1", "precision", "recall"],
        return_train_score=True
    )

    f1_train_scores.append(cv_results["train_f1"].mean())
    f1_cv_scores.append(cv_results["test_f1"].mean())
    f1_std_scores.append(np.std(f1_cv_scores))
    prec_train_scores.append(cv_results["train_precision"].mean())
    prec_cv_scores.append(cv_results["test_precision"].mean())
    rec_train_scores.append(cv_results["train_recall"].mean())
    rec_cv_scores.append(cv_results["test_recall"].mean())

plt.semilogx(C_vals, f1_train_scores, label="f1_train")
plt.semilogx(C_vals, f1_cv_scores, label="f1_valid")
plt.legend()
plt.xlabel("C")
plt.ylabel("F1 Scores");
```



In [153…

```
from sklearn.metrics import precision_recall_curve, average_precision_score

pipe = make_pipeline(
    preprocessor,
```

```
    LogisticRegression(max_iter=100, C=10, random_state=123, class_weight=None)
)
pipe.fit(X_train, y_train)

probs = pipe.predict_proba(X_train)[:, 1]
precisions, recalls, thresholds = precision_recall_curve(y_train, probs)
ap_score = average_precision_score(y_train, probs)

plt.plot(recalls, precisions, label=f'PR Curve')
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.legend(loc='upper right')
plt.show()
```



In [154…

```
pd.DataFrame({"C": C_vals, "f1_train": f1_train_scores, "f1_cv": f1_cv_scores, "f1_
```

Out[154…

	C	f1_train	f1_cv	f1_std	precision_train	precision_cv	recall_train	recall_cv
0	0.001	0.363061	0.360926	0.000000	0.681666	0.680366	0.247440	0.245952
1	0.010	0.407839	0.405397	0.022236	0.667049	0.664867	0.293728	0.291811
2	0.100	0.415670	0.413051	0.022982	0.660007	0.658137	0.303434	0.301197
3	1.000	0.417645	0.415081	0.022057	0.659117	0.657066	0.305727	0.303543
4	5.000	0.417741	0.415491	0.020851	0.658844	0.657092	0.305887	0.303970
5	10.000	0.417435	0.415730	0.019711	0.658569	0.657261	0.305620	0.304183
6	100.000	0.417555	0.415966	0.018702	0.658647	0.657396	0.305727	0.304396

The best hyperparameter value for C was 10, as seen by the graph beginning to plateau for train and valid. The best hyperparameter value for max_iter was 100.

Overall, it seems like Logistic Regression isn't affected much by hyperparameter tuning. This makes sense as non-linear relationships between different features are really difficult for this model to predict an accurate outcome.

This inability to predict trends in the data is especially obvious when we set class_weight='balanced' to offset the default imbalance.

In [155…

```
C_vals = [0.001, 0.01, 0.1, 1, 5, 10, 100]
Max_iter = [1, 10, 100, 10000]
pipe_lr = make_pipeline(preprocessor, LogisticRegression(random_state=123, class_we
param_grid = {
    "logisticregression__C": C_vals,
    "logisticregression__max_iter" : Max_iter
}

grid_search = GridSearchCV(pipe_lr, param_grid=param_grid, n_jobs=-1, scoring='f1')
grid_search.fit(X_train, y_train)
best_C =  grid_search.best_params_.get("logisticregression__C")
best_Max_iter = grid_search.best_params_.get("logisticregression__max_iter")
best_score = grid_search.best_score_
best_C
best_Max_iter
```

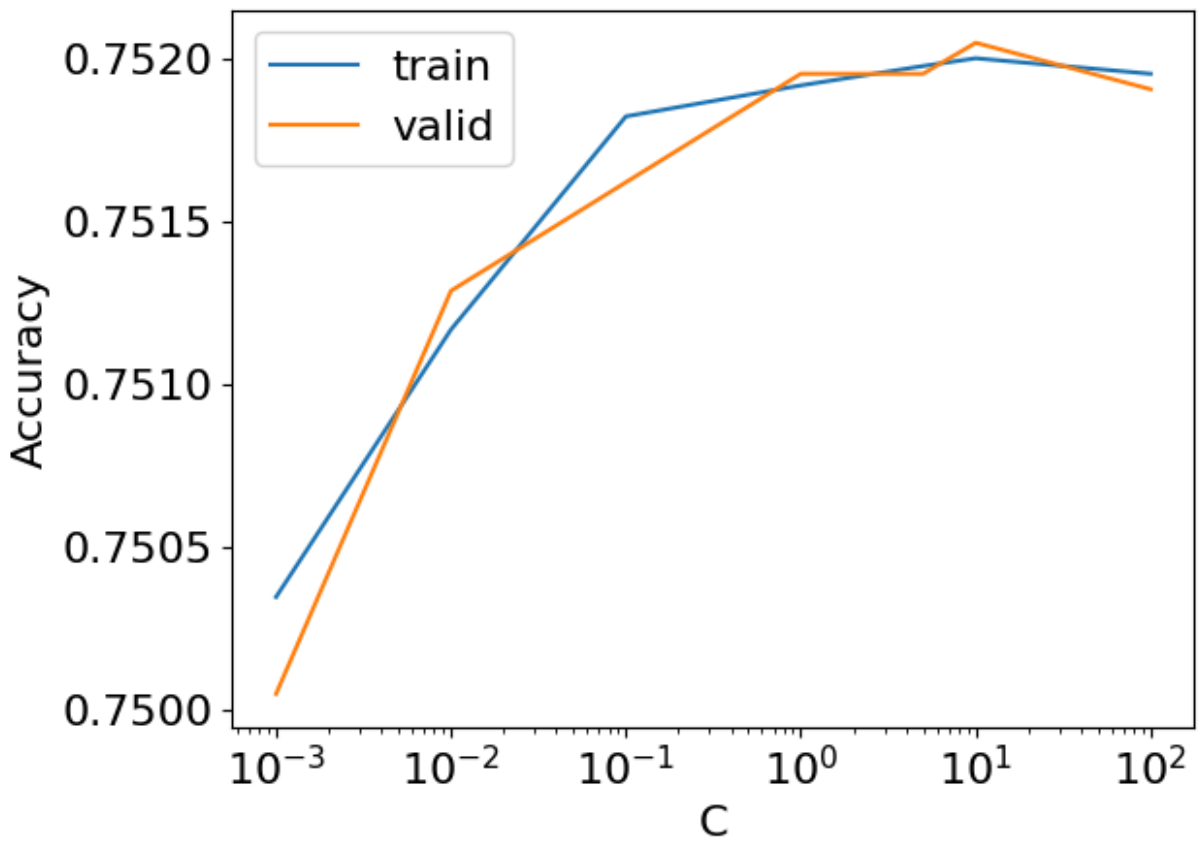
Out[155…

```
100
```

In [156…

```
# taken from hw4
bal_train_scores = []
bal_cv_scores = []
bal_std_scores = []
bal_fit_times = []
bal_score_times = []

for C in C_vals:
    #     print(C)
```

```
    pipe = make_pipeline(
        preprocessor,
        LogisticRegression(max_iter=100, C=C, random_state=123, class_weight='balan
    )
    cv_results = cross_validate(pipe, X_train, y_train, return_train_score=True)

    bal_train_scores.append(cv_results["train_score"].mean())
    bal_cv_scores.append(cv_results["test_score"].mean())
    bal_std_scores.append(np.std(cv_scores))
    bal_fit_times.append(cv_results["fit_time"].mean())
    bal_score_times.append(cv_results["score_time"].mean())

plt.semilogx(C_vals, bal_train_scores, label="train")
plt.semilogx(C_vals, bal_cv_scores, label="valid")
plt.legend()
plt.xlabel("C")
plt.ylabel("Accuracy");
```



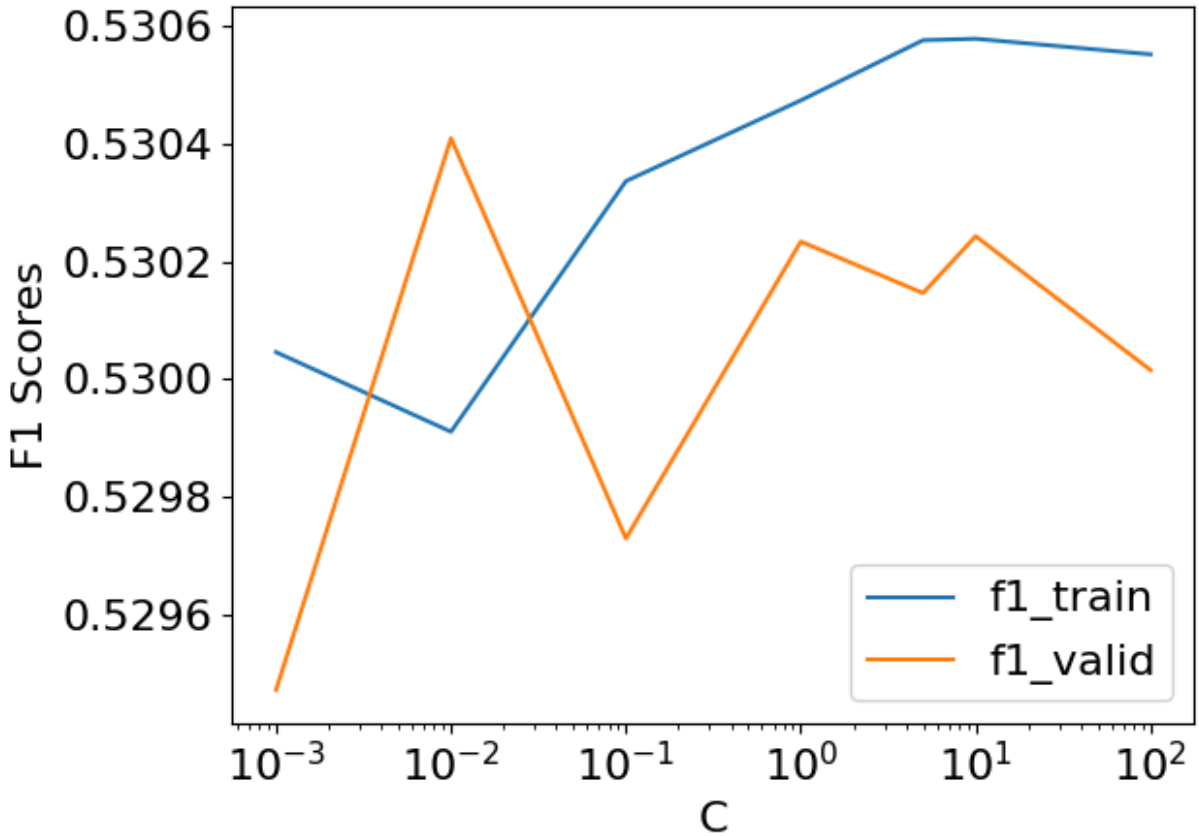
In [79]:
```
pd.DataFrame({"C": C_vals, "train": bal_train_scores, "cv": bal_cv_scores, "std": s
```

Out[79]:

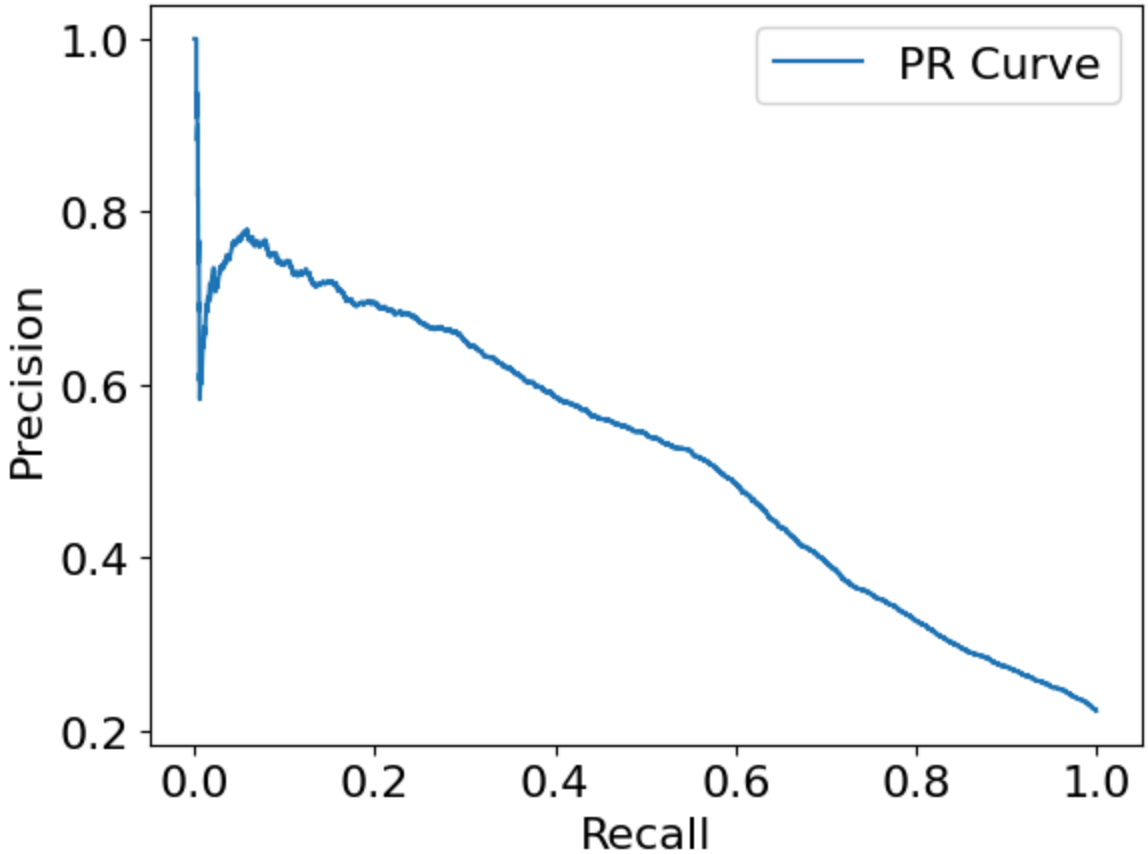
	C	train	cv	std	fit_times	score_times
0	0.001	0.750345	0.750048	0.000000	0.056181	0.006171
1	0.010	0.751167	0.751286	0.001643	0.063018	0.006327
2	0.100	0.751821	0.751619	0.001538	0.083683	0.006232
3	1.000	0.751917	0.751952	0.001423	0.086496	0.006412
4	5.000	0.751976	0.751952	0.001325	0.086269	0.006687
5	10.000	0.752000	0.752048	0.001244	0.086168	0.006363
6	100.000	0.751952	0.751905	0.001177	0.084754	0.006151

In [32]:

```
# taken from hw4
f1_bal_train_scores = []
f1_bal_cv_scores = []
f1_bal_std_scores = []
prec_bal_train_scores = []
prec_bal_cv_scores = []
rec_bal_train_scores = []
rec_bal_cv_scores = []
acc_bal_train_scores = []
acc_bal_test_scores = []

for C in C_vals:
    #     print(C)
    pipe = make_pipeline(
        preprocessor,
        LogisticRegression(max_iter=100, C=C, random_state=123, class_weight='balan
    )
    cv_results = cross_validate(
        pipe, X_train, y_train,
        scoring=["accuracy","f1", "precision", "recall"],
        return_train_score=True
    )

    f1_bal_train_scores.append(cv_results["train_f1"].mean())
    f1_bal_cv_scores.append(cv_results["test_f1"].mean())
    f1_bal_std_scores.append(np.std(f1_cv_scores))
    prec_bal_train_scores.append(cv_results["train_precision"].mean())
    prec_bal_cv_scores.append(cv_results["test_precision"].mean())
    rec_bal_train_scores.append(cv_results["train_recall"].mean())
    rec_bal_cv_scores.append(cv_results["test_recall"].mean())
    acc_bal_train_scores.append(cv_results["train_accuracy"].mean())
    acc_bal_test_scores.append(cv_results["test_accuracy"].mean())

plt.semilogx(C_vals, f1_bal_train_scores, label="f1_train")
plt.semilogx(C_vals, f1_bal_cv_scores, label="f1_valid")
plt.legend()
plt.xlabel("C")
plt.ylabel("F1 Scores");
```



In [33]:
```
from sklearn.metrics import precision_recall_curve, average_precision_score

pipe = make_pipeline(
    preprocessor,
    LogisticRegression(max_iter=100, C=10, random_state=123, class_weight='balanced
)
pipe.fit(X_train, y_train)

probs = pipe.predict_proba(X_train)[:, 1]
precisions, recalls, thresholds = precision_recall_curve(y_train, probs)
ap_score = average_precision_score(y_train, probs)

plt.plot(recalls, precisions, label=f'PR Curve')
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.legend(loc='upper right')
plt.show()
```



In [34]:
```
pd.DataFrame({"C": C_vals, "accuracy: train": acc_bal_train_scores, "accuracy: test
```

Out[34]:

	C	accuracy: train	accuracy: test	f1_train	f1_cv	f1_std	precision_train	precision_cv
0	0.001	0.750345	0.750048	0.530046	0.529471	0.018702	0.457126	0.456852
1	0.010	0.751167	0.751286	0.529910	0.530409	0.018702	0.458193	0.458571
2	0.100	0.751821	0.751619	0.530336	0.529729	0.018702	0.459143	0.458919
3	1.000	0.751917	0.751952	0.530474	0.530234	0.018702	0.459292	0.459447
4	5.000	0.751976	0.751952	0.530576	0.530146	0.018702	0.459389	0.459427
5	10.000	0.752000	0.752048	0.530579	0.530243	0.018702	0.459421	0.459574
6	100.000	0.751952	0.751905	0.530552	0.530015	0.018702	0.459353	0.459353

On balanced data the f1 scores get pretty weird across different C values and the overall accuracy of the model becomes worse than DummyClassifier. For balanced data, the best C value is 0.01 and the best value of max_iter is 100, resulting in an f1 score of 0.530409.

However, the actual ratio (F1 score) between precision and recall remains relatively the same, as seen by their graphs. Despite the overall accuracy of the model decreasing, The recall score seems to increase by a lot when the data is balanced, whereas precision decreases,

which is better for our usecase (as discussed in our EDA). The erratic behaviour in the graphs instead of a gradual increase and plateau also suggests that when data is imbalanced logistic regression is likely predicting patterns randomly, which is sometimes randomly leading to high validation scores. Overall, despite balancing the data leading to a better recall score, we can't be sure whether logistic regression alone is good enough to solve this problem.

8. Different models

---

rubric={points:12}

Your tasks:

1. **Experiment with additional models**: Train at least **three models** other than a linear model. Ensure that **at least one** of these models is a **tree-based ensemble model** (e.g., Random Forest, Gradient Boosting, or XGBoost).
2. **Compare and interpret results**: Summarize your findings in terms of **overfitting/underfitting** behavior and **fit/score times** for each model. Reflect on your results. Were you able to **outperform the linear model**?

Solution_8

Points: 12

In [ ]:
```
#dt
pipe_dt = make_pipeline(preprocessor, DecisionTreeClassifier(random_state=123))
scores_dt = cross_validate(pipe_dt, X_train, y_train, scoring=["accuracy", "f1", "p
pd.DataFrame(scores_dt).mean()
```

Out[ ]:
```
fit_time           0.265005
score_time         0.011792
test_accuracy      0.726143
train_accuracy     0.999405
test_f1            0.404237
train_f1           0.998665
test_precision     0.393122
train_precision    1.000000
test_recall        0.416169
train_recall       0.997334
dtype: float64
```

A basic, unoptimized decision tree classifier seems to perform worse than the linear model. It is overfitting quite a bit, shown by the near perfect train scores and relatively low test scores.

Its score and fit times also are slightly higher than the linear model.

In [36]:
```
#knn
pipe_knn = make_pipeline(preprocessor, KNeighborsClassifier())
scores_knn = cross_validate(pipe_knn, X_train, y_train, scoring=["accuracy", "f1",
pd.DataFrame(scores_knn).mean()
```

Out[36]:
```
fit_time           0.068640
score_time         0.500967
test_accuracy      0.793952
train_accuracy     0.843488
test_f1            0.439982
train_f1           0.577027
test_precision     0.559366
train_precision    0.727206
test_recall        0.362625
train_recall       0.478295
dtype: float64
```

The KNN model outperforms the logistic regression model on accuracy, but is worse in terms of f1 score, with a low recall and high precision. It doesn't seem to be extremely overfitted, as the difference between the train and test accuracy isn't very large. The fit time was lower, but the score time was significantly higher.

In [ ]:
```
from sklearn.ensemble import RandomForestClassifier
#random forest
pipe_rf = make_pipeline(preprocessor, RandomForestClassifier(random_state=123, n_jo
scores_rf = cross_validate(pipe_rf, X_train, y_train, scoring=["accuracy", "f1", "p
pd.DataFrame(scores_rf).mean()
```

Out[ ]:
```
fit_time           0.373739
score_time         0.049628
test_accuracy      0.816143
train_accuracy     0.999381
test_f1            0.477906
train_f1           0.998613
test_precision     0.653569
train_precision    0.999146
test_recall        0.376920
train_recall       0.998080
dtype: float64
```

A basic, unoptimized random forest classifier seems to outperform the linear model in terms of accuracy. However, it does seem to underperform for f1 score, and recall. It also seems to be overfitting, seeing how all the train scores are near perfect even though the test scores are lower. Its fit and score times also seem to be larger than linear regression's.

9. Feature selection

---

rubric={points:2}

Your tasks:

1. **Perform feature selection**: Attempt to select relevant features using methods such as `RFECV` or forward selection.
2. **Evaluate the impact** Compare the model performance before and after feature selection. Do the results improve with feature selection?
3. **Summarize findings** Summarize your observations and decide whether to **keep feature selection** in your pipeline. If it improves results, retain it for the next exercises; otherwise, you may choose to omit it.

Solution_9

Points: 2

KNN does not have coefficients, so we will be using RFE on our decision tree and random forest models instead.

In [96]:
```
rfe_dt = RFECV(DecisionTreeClassifier(random_state=123), cv=10)
rfe_dt.fit(X_train_pp, y_train)
print(rfe_dt.support_)
print(X_train_pp.columns[rfe_dt.support_])

rfe_dt_pipe = make_pipeline(preprocessor, RFECV(DecisionTreeClassifier(random_state
pd.DataFrame(cross_validate(rfe_dt_pipe, X_train, y_train, return_train_score=True,
```

```
[ True  True  True  True  True False  True  True  True  True  True  True
  True  True  True  True  True  True  True  True  True  True False  True
  True  True  True  True  True  True  True False  True  True  True]
Index(['pipeline-1__LIMIT_BAL', 'pipeline-1__AGE', 'pipeline-1__PAY_0',
       'pipeline-1__PAY_2', 'pipeline-1__PAY_3', 'pipeline-1__PAY_5',
       'pipeline-1__PAY_6', 'pipeline-1__BILL_AMT1', 'pipeline-1__BILL_AMT2',
       'pipeline-1__BILL_AMT3', 'pipeline-1__BILL_AMT4',
       'pipeline-1__BILL_AMT5', 'pipeline-1__BILL_AMT6',
       'pipeline-1__PAY_AMT1', 'pipeline-1__PAY_AMT2', 'pipeline-1__PAY_AMT3',
       'pipeline-1__PAY_AMT4', 'pipeline-1__PAY_AMT5', 'pipeline-1__PAY_AMT6',
       'pipeline-1__BILL_PAY1', 'pipeline-1__BILL_PAY2',
       'pipeline-1__BILL_PAY4', 'pipeline-1__BILL_PAY5',
       'pipeline-1__BILL_PAY6', 'pipeline-1__MAX_DELAY',
       'pipeline-1__NUM_DELAY', 'pipeline-1__AVG_DELAY',
       'ordinalencoder__EDUCATION', 'onehotencoder__SEX_2',
       'pipeline-2__MARRIAGE_1', 'pipeline-2__MARRIAGE_2',
       'pipeline-2__MARRIAGE_3'],
      dtype='object')
```

Out[96]:
```
fit_time           55.659072
score_time          0.013034
test_accuracy       0.724286
train_accuracy      0.995036
test_f1             0.391418
train_f1            0.988564
test_precision      0.385781
train_precision     0.999232
test_recall         0.397400
train_recall        0.978511
dtype: float64
```

The decision tree model chose to get rid of three features, onehotencoder__SEX_1, pipeline-1__PAY_4, and pipeline-1__BILL_PAY3. Overall, almost every single score actually decreased by 0.1-1% after feature removal, with test recall and test f1 specifically decreasing around 1%. This is surprising, as even training scores like train accuracy decreased.

In [95]:
```
# ensemble
rfe_rf = RFECV(RandomForestClassifier(random_state=123), cv=10)
rfe_rf.fit(X_train_pp, y_train)
print(rfe_rf.support_)
print(X_train_pp.columns[rfe_rf.support_])

rfe_rf_pipe = make_pipeline(preprocessor, RFECV(RandomForestClassifier(random_state
pd.DataFrame(cross_validate(rfe_rf_pipe, X_train, y_train, return_train_score=True,
```

```
[ True  True  True  True  True  True  True  True  True  True  True  True
  True  True  True  True  True  True  True  True  True  True  True  True
  True  True  True  True  True  True  True False  True  True  True]
Index(['pipeline-1__LIMIT_BAL', 'pipeline-1__AGE', 'pipeline-1__PAY_0',
       'pipeline-1__PAY_2', 'pipeline-1__PAY_3', 'pipeline-1__PAY_4',
       'pipeline-1__PAY_5', 'pipeline-1__PAY_6', 'pipeline-1__BILL_AMT1',
       'pipeline-1__BILL_AMT2', 'pipeline-1__BILL_AMT3',
       'pipeline-1__BILL_AMT4', 'pipeline-1__BILL_AMT5',
       'pipeline-1__BILL_AMT6', 'pipeline-1__PAY_AMT1', 'pipeline-1__PAY_AMT2',
       'pipeline-1__PAY_AMT3', 'pipeline-1__PAY_AMT4', 'pipeline-1__PAY_AMT5',
       'pipeline-1__PAY_AMT6', 'pipeline-1__BILL_PAY1',
       'pipeline-1__BILL_PAY2', 'pipeline-1__BILL_PAY3',
       'pipeline-1__BILL_PAY4', 'pipeline-1__BILL_PAY5',
       'pipeline-1__BILL_PAY6', 'pipeline-1__MAX_DELAY',
       'pipeline-1__NUM_DELAY', 'pipeline-1__AVG_DELAY',
       'ordinalencoder__EDUCATION', 'onehotencoder__SEX_2',
       'pipeline-2__MARRIAGE_1', 'pipeline-2__MARRIAGE_2',
       'pipeline-2__MARRIAGE_3'],
      dtype='object')
```

Out[95]:
```
fit_time           153.707826
score_time           0.073196
test_accuracy        0.814857
train_accuracy       0.998810
test_f1              0.479028
train_f1             0.997332
test_precision       0.644328
train_precision      0.997971
test_recall          0.381399
train_recall         0.996694
dtype: float64
```

The random forest model chose to remove one feature, onehotencoder__SEX_1. The model test accuracy, test f1 score, and test recall score all increased by about 1%, while test precision decreased by about 1%. The training scores did not have a noticeable difference, as the model is so overfit that it couldn't increase the score much higher. Overall, RFE did not have a significant impact on the effectiveness of the random forest model.

Looking at the resuls of running feature selection on both models, neither resulted in great results. Decision tree actually had a unanimously negative impact on the scores, while random forest had an extremely small positive impact on a select few scores. Neither change is worth the time cost of running RFE in the pipeline, with decision tree taking 8 minutes to run and random forest taking an excruciating 28 minutes, despite running it on a heavy duty laptop with n_jobs being set to -1. For these reasons we will not be using feature selection in our pipelines.

10. Hyperparameter optimization

---

rubric={points:10}

Your tasks:

1. **Optimize hyperparameters**: Attempt to optimize hyperparameters for the models you have tried so far. In at least **one case**, tune **multiple hyperparameters** for a single model.

2. **Use suitable optimization methods**: You may use any of the following approaches for hyperparameter optimization:

   - `GridSearchCV`
   - `RandomizedSearchCV`
   - Bayesian optimization with scikit-optimize

3. **Summarize your results**: Report and compare the optimized results across models. Discuss whether hyperparameter optimization led to performance improvements.

Solution_10

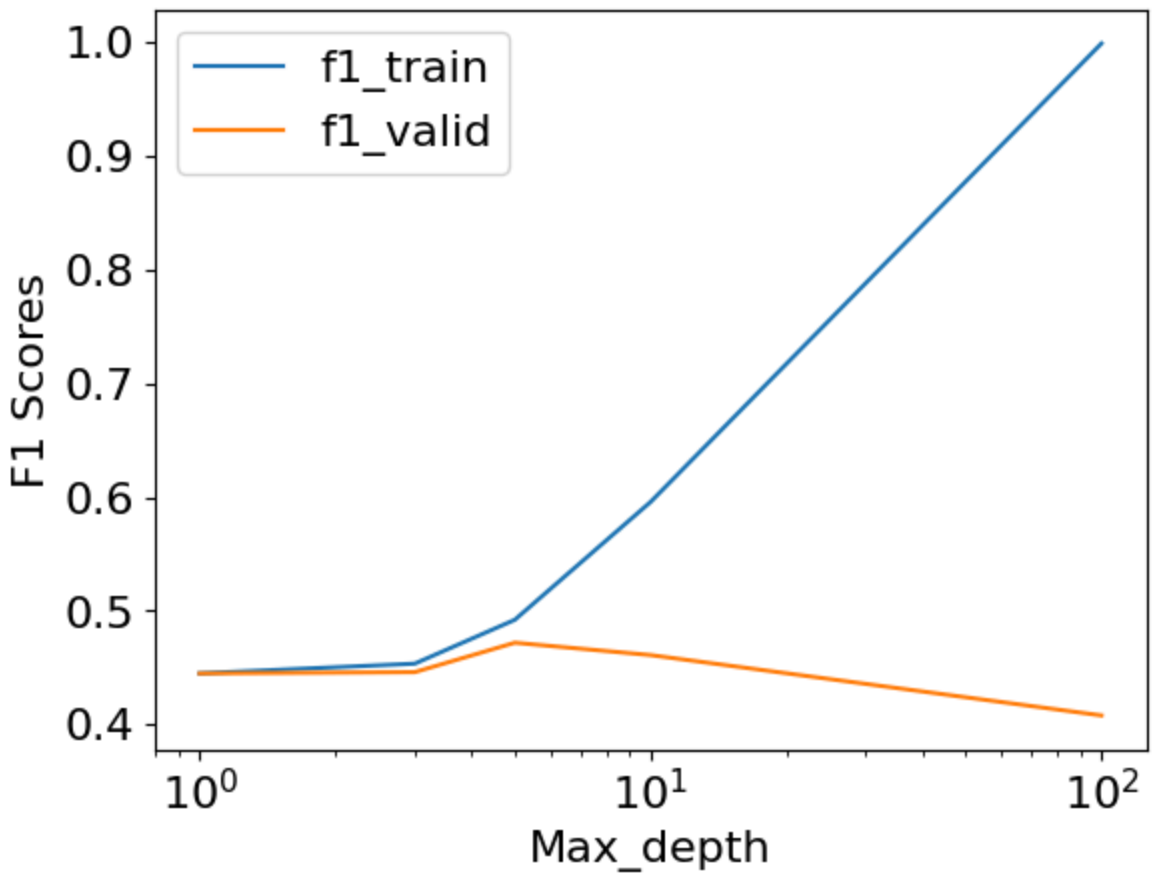
Points: 10

Type your answer here, replacing this text.

In [101…

```
#dt
# taken from hw4
depth = [1, 3, 5, 10, 100]

dt_train_scores = []
dt_cv_scores = []
dt_std_scores = []
f1_dt_train_scores = []
f1_dt_cv_scores = []
f1_dt_std_scores = []
prec_dt_train_scores = []
prec_dt_cv_scores = []
rec_dt_train_scores = []
rec_dt_cv_scores = []

for n in depth:
    #     print(C)
    pipe = make_pipeline(
        preprocessor,
        DecisionTreeClassifier(random_state=123, max_depth=n),
    )
    cv_results = cross_validate(
        pipe, X_train, y_train,
        scoring=["accuracy", "f1", "precision", "recall"],
        return_train_score=True
    )

    dt_train_scores.append(cv_results["train_accuracy"].mean())
    dt_cv_scores.append(cv_results["test_accuracy"].mean())
    dt_std_scores.append(np.std(dt_cv_scores))
    f1_dt_train_scores.append(cv_results["train_f1"].mean())
    f1_dt_cv_scores.append(cv_results["test_f1"].mean())
    f1_dt_std_scores.append(np.std(f1_cv_scores))
    prec_dt_train_scores.append(cv_results["train_precision"].mean())
    prec_dt_cv_scores.append(cv_results["test_precision"].mean())
    rec_dt_train_scores.append(cv_results["train_recall"].mean())
    rec_dt_cv_scores.append(cv_results["test_recall"].mean())

plt.semilogx(depth, f1_dt_train_scores, label="f1_train")
plt.semilogx(depth, f1_dt_cv_scores, label="f1_valid")
plt.legend()
```

```
plt.xlabel("Max_depth")
plt.ylabel("F1 Scores");
```



In [99]:
```
#dt
# taken from lecture 8, CPSC 330 2025W2
param_grid = {
    "decisiontreeclassifier__max_depth": depth
}

pipe_dt = make_pipeline(preprocessor, DecisionTreeClassifier(random_state=123))

grid_search = GridSearchCV(pipe_dt, param_grid=param_grid, n_jobs=-1, scoring="f1")
grid_search.fit(X_train,y_train)

best_depth = grid_search.best_params_.get("decisiontreeclassifier__max_depth")
best_score = grid_search.best_score_
print(best_depth)
print(best_score)
```

```
5
0.4718677626330153
```

In [100…
```
#dt
pipe_dt = make_pipeline(preprocessor, DecisionTreeClassifier(random_state=123, max_

pd.DataFrame({"depth": depth,
                "accuracy_train": dt_train_scores,
                "accuracy_cv": dt_cv_scores,
                "dt_std": dt_std_scores,
```

```
                "f1_train": f1_dt_train_scores,
                "f1_cv": f1_dt_cv_scores,
                "f1_std": f1_dt_std_scores,
                "precision_train": prec_dt_train_scores,
                "precision_cv": prec_dt_cv_scores,
                "recall_train": rec_dt_train_scores,
                "recall_cv": rec_dt_cv_scores})
```

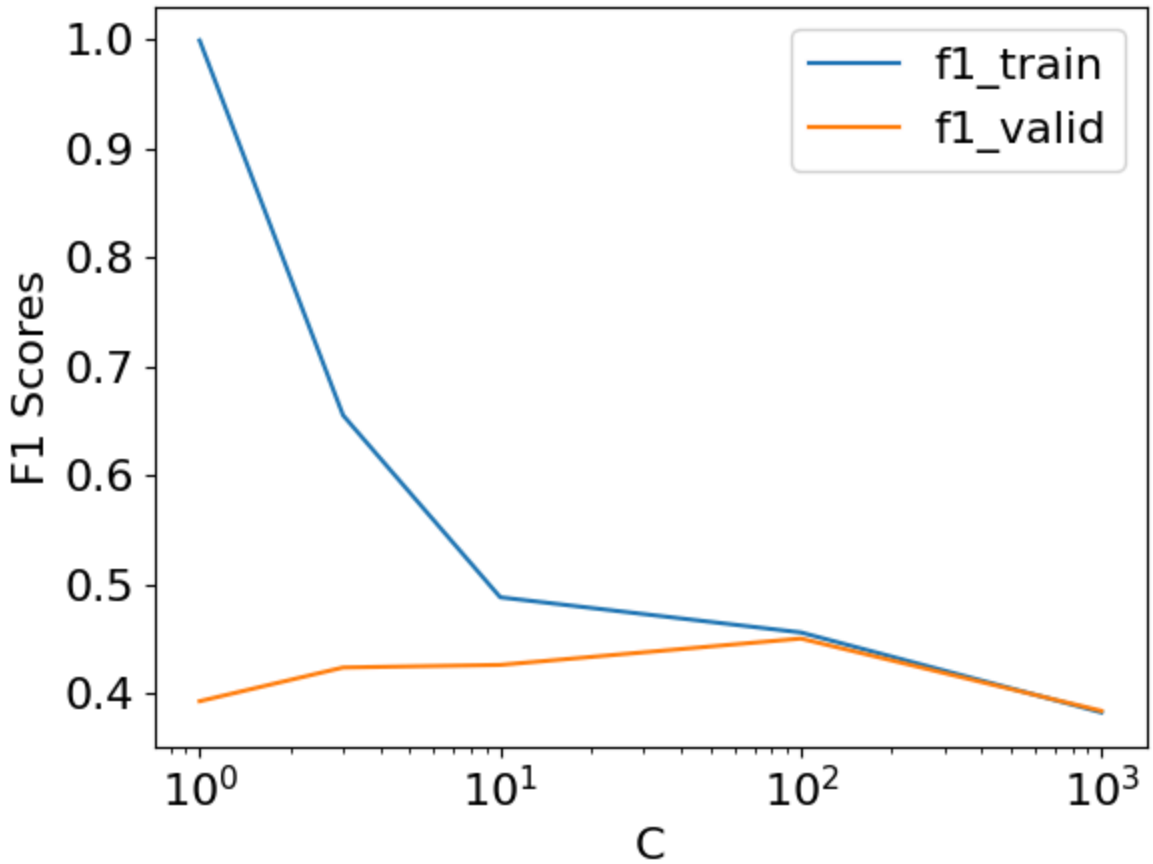
Out[100…

	depth	accuracy_train	accuracy_cv	dt_std	f1_train	f1_cv	f1_std	precision_tra
0	1	0.818381	0.818381	0.000000	0.445152	0.445097	0.018702	0.6999
1	5	0.819917	0.818095	0.000143	0.453676	0.446214	0.018702	0.7047
2	10	0.825060	0.818095	0.000135	0.492210	0.471797	0.018702	0.7004
3	100	0.854393	0.806095	0.005239	0.595698	0.462817	0.018702	0.7845
4	300	0.999405	0.724857	0.036426	0.998665	0.402715	0.018702	1.0000

In [44]:

```
#knn
# taken from hw4
n_neighbors = [1, 3, 10, 100, 1000]

knn_train_scores = []
knn_cv_scores = []
knn_std_scores = []
f1_knn_train_scores = []
f1_knn_cv_scores = []
f1_knn_std_scores = []
prec_knn_train_scores = []
prec_knn_cv_scores = []
rec_knn_train_scores = []
rec_knn_cv_scores = []

for n in n_neighbors:
    #     print(C)
    pipe = make_pipeline(
        preprocessor,
        KNeighborsClassifier(n_neighbors=n),
    )
    cv_results = cross_validate(
        pipe, X_train, y_train,
        scoring=["f1", "precision", "recall", "accuracy"],
        return_train_score=True
    )

    knn_train_scores.append(cv_results["train_accuracy"].mean())
    knn_cv_scores.append(cv_results["test_accuracy"].mean())
    knn_std_scores.append(np.std(knn_cv_scores))
    f1_knn_train_scores.append(cv_results["train_f1"].mean())
    f1_knn_cv_scores.append(cv_results["test_f1"].mean())
    f1_knn_std_scores.append(np.std(f1_cv_scores))
    prec_knn_train_scores.append(cv_results["train_precision"].mean())
    prec_knn_cv_scores.append(cv_results["test_precision"].mean())
```

```
    rec_knn_train_scores.append(cv_results["train_recall"].mean())
    rec_knn_cv_scores.append(cv_results["test_recall"].mean())

plt.semilogx(n_neighbors, f1_knn_train_scores, label="f1_train")
plt.semilogx(n_neighbors, f1_knn_cv_scores, label="f1_valid")
plt.legend()
plt.xlabel("C")
plt.ylabel("F1 Scores");
```



In [73]:

```
#knn
# taken from lecture 8, CPSC 330 2025W2
param_grid = {
    "kneighborsclassifier__n_neighbors": n_neighbors
}

pipe_knn = make_pipeline(preprocessor, KNeighborsClassifier())

grid_search = GridSearchCV(pipe_knn, param_grid=param_grid, n_jobs=-1, scoring="f1"
grid_search.fit(X_train,y_train)

best_neighbors = grid_search.best_params_.get("kneighborsclassifier__n_neighbors")
best_score = grid_search.best_score_
print(best_neighbors)
print(best_score)
```

```
100
0.4501581794897301
```

In [46]:
```
#knn
pipe_knn = make_pipeline(preprocessor, KNeighborsClassifier(n_neighbors=100))

pd.DataFrame({"n_neighbors": n_neighbors,
                "accuracy_train": knn_train_scores,
                "accuracy_cv": knn_cv_scores,
                "knn_std": knn_std_scores,
                "f1_train": f1_knn_train_scores,
                "f1_cv": f1_knn_cv_scores,
                "f1_std": f1_knn_std_scores,
                "precision_train": prec_knn_train_scores,
                "precision_cv": prec_knn_cv_scores,
                "recall_train": rec_knn_train_scores,
                "recall_cv": rec_knn_cv_scores})
```

Out[46]:

	n_neighbors	accuracy_train	accuracy_cv	knn_std	f1_train	f1_cv	f1_std	precisi
0	1	0.999405	0.727810	0.000000	0.998666	0.392709	0.018702	(
1	3	0.866000	0.774667	0.023429	0.654933	0.423607	0.018702	(
2	10	0.828631	0.808810	0.033204	0.488002	0.426002	0.018702	(
3	100	0.815869	0.814286	0.034460	0.455765	0.450158	0.018702	(
4	1000	0.805369	0.806095	0.032367	0.382404	0.383844	0.018702	(

In [153…
```
#random forest
depth = [1, 5, 10, 100, 300]
estimators = [1, 5, 10, 100, 300]

param_grid = {
    "randomforestclassifier__max_depth": depth,
    "randomforestclassifier__n_estimators": estimators
}

pipe_rf = make_pipeline(preprocessor, RandomForestClassifier())

grid_search = GridSearchCV(pipe_rf, param_grid=param_grid, n_jobs=-1, scoring="f1")
grid_search.fit(X_train,y_train)

best_depth = grid_search.best_params_.get("randomforestclassifier__max_depth")
best_estimators = grid_search.best_params_.get("randomforestclassifier__n_estimator
best_score = grid_search.best_score_
print(best_depth)
print(best_estimators)
print(best_score)
```

```
100
300
0.4811049563064887
```

In [152…
```
from itertools import product
#random forest
```

```
rf_train_scores = []
rf_cv_scores = []
rf_std_scores = []
f1_rf_train_scores = []
f1_rf_cv_scores = []
f1_rf_std_scores = []
prec_rf_train_scores = []
prec_rf_cv_scores = []
rec_rf_train_scores = []
rec_rf_cv_scores = []

for d in depth:
    for e in estimators:
        pipe = make_pipeline(
            preprocessor,
            RandomForestClassifier(n_estimators=e, max_depth=d, n_jobs=-1),
        )
        cv_results = cross_validate(
            pipe, X_train, y_train,
            scoring=["f1", "precision", "recall", "accuracy"],
            return_train_score=True
        )

        rf_train_scores.append(cv_results["train_accuracy"].mean())
        rf_cv_scores.append(cv_results["test_accuracy"].mean())
        rf_std_scores.append(np.std(knn_cv_scores))
        f1_rf_train_scores.append(cv_results["train_f1"].mean())
        f1_rf_cv_scores.append(cv_results["test_f1"].mean())
        f1_rf_std_scores.append(np.std(f1_cv_scores))
        prec_rf_train_scores.append(cv_results["train_precision"].mean())
        prec_rf_cv_scores.append(cv_results["test_precision"].mean())
        rec_rf_train_scores.append(cv_results["train_recall"].mean())
        rec_rf_cv_scores.append(cv_results["test_recall"].mean())

rf_results = pd.DataFrame(grid_search.cv_results_)

pipe_rf = make_pipeline(preprocessor, RandomForestClassifier(n_estimators=best_esti

combinations = list(product(depth, estimators))
depth_col, estimators_col = zip(*combinations)
pd.DataFrame({"max_depth": depth_col,
                "n_estimators": estimators_col,
                "accuracy_train": rf_train_scores,
                "accuracy_cv": rf_cv_scores,
                "rf_std": rf_std_scores,
                "f1_train": f1_rf_train_scores,
                "f1_cv": f1_rf_cv_scores,
                "f1_std": f1_rf_std_scores,
                "precision_train": prec_rf_train_scores,
                "precision_cv": prec_rf_cv_scores,
                "recall_train": rec_rf_train_scores,
                "recall_cv": rec_rf_cv_scores})
```

```
c:\Users\justh\miniforge3\envs\cpsc330\Lib\site-packages\sklearn\metrics\_classifica
tion.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0
due to no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\justh\miniforge3\envs\cpsc330\Lib\site-packages\sklearn\metrics\_classifica
tion.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0
due to no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\justh\miniforge3\envs\cpsc330\Lib\site-packages\sklearn\metrics\_classifica
tion.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0
due to no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\justh\miniforge3\envs\cpsc330\Lib\site-packages\sklearn\metrics\_classifica
tion.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0
due to no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\justh\miniforge3\envs\cpsc330\Lib\site-packages\sklearn\metrics\_classifica
tion.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0
due to no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\justh\miniforge3\envs\cpsc330\Lib\site-packages\sklearn\metrics\_classifica
tion.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0
due to no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\justh\miniforge3\envs\cpsc330\Lib\site-packages\sklearn\metrics\_classifica
tion.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0
due to no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\justh\miniforge3\envs\cpsc330\Lib\site-packages\sklearn\metrics\_classifica
tion.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0
due to no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\justh\miniforge3\envs\cpsc330\Lib\site-packages\sklearn\metrics\_classifica
tion.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0
due to no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\justh\miniforge3\envs\cpsc330\Lib\site-packages\sklearn\metrics\_classifica
tion.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0
due to no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\justh\miniforge3\envs\cpsc330\Lib\site-packages\sklearn\metrics\_classifica
tion.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0
due to no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

Out[152…

	max_depth	n_estimators	accuracy_train	accuracy_cv	rf_std	f1_train	f1_cv	
0	1	1	0.804107	0.800857	0.032381	0.336489	0.328692	0.0
1	1	5	0.792548	0.791238	0.032381	0.183022	0.175358	0.0
2	1	10	0.787655	0.787619	0.032381	0.120920	0.118892	0.0
3	1	100	0.783405	0.783429	0.032381	0.077469	0.076939	0.0
4	1	300	0.790750	0.789810	0.032381	0.153336	0.148651	0.0
5	5	1	0.815429	0.809571	0.032381	0.449015	0.430754	0.0
6	5	5	0.821857	0.818476	0.032381	0.474923	0.466203	0.0
7	5	10	0.822060	0.817333	0.032381	0.471903	0.459426	0.0
8	5	100	0.824119	0.818905	0.032381	0.484965	0.468145	0.0
9	5	300	0.823964	0.817524	0.032381	0.480686	0.461008	0.0
10	10	1	0.831690	0.796810	0.032381	0.535781	0.435865	0.0
11	10	5	0.852595	0.810429	0.032381	0.579634	0.461857	0.0
12	10	10	0.858190	0.814857	0.032381	0.591141	0.462277	0.0
13	10	100	0.859619	0.818048	0.032381	0.594086	0.473845	0.0
14	10	300	0.859476	0.818095	0.032381	0.593677	0.475194	0.0
15	100	1	0.899798	0.726333	0.032381	0.778104	0.398248	0.0
16	100	5	0.970548	0.788048	0.032381	0.931826	0.447122	0.0
17	100	10	0.978690	0.805286	0.032381	0.950107	0.433590	0.0
18	100	100	0.999393	0.813524	0.032381	0.998639	0.474757	0.0
19	100	300	0.999405	0.816048	0.032381	0.998666	0.480568	0.0
20	300	1	0.898345	0.723905	0.032381	0.774190	0.390815	0.0
21	300	5	0.970976	0.786571	0.032381	0.932802	0.438071	0.0
22	300	10	0.978631	0.804429	0.032381	0.950005	0.430751	0.0
23	300	100	0.999381	0.815190	0.032381	0.998613	0.477460	0.0
24	300	300	0.999405	0.815762	0.032381	0.998666	0.479486	0.0

Hyperparameter optimization helped slightly with model performance. Decision tree's f1 score increased the most dramatically by 7%, KNN increased by 1%, and random forest increased by 0.3%. For decision trees, it mainly helped to prevent overfitting, which also led to its highest test accuracy of 81.18%. For KNN, it was able to find the model's largest f1 score and accuracy. Due to KNN's model, there wasn't really any hyperparameter where

recall could be prioritized over precision without the model being grossly underfit, so it unfortunately had a lower recall score. This issue is echoed in decision tree and random forest as well. Lastly, random forest had the best f1 score out of the three models after hyperparameter tuning, with an f1 score of 48%. Random forest had very similar accuracies to decision tree, with the max being around 81.18%

11. Interpretation and feature importances

rubric={points:10}

Your tasks:

1. **Interpret model feature importance**: Use one of the interpretation methods discussed in class (e.g., `shap`), or another suitable method of your choice, to examine the most important features of one of your **non-linear models**.
2. **Summarize insights**: Summarize your observations about which features contribute most to the model's predictions and how they influence the outcomes.

Solution_11

Points: 10

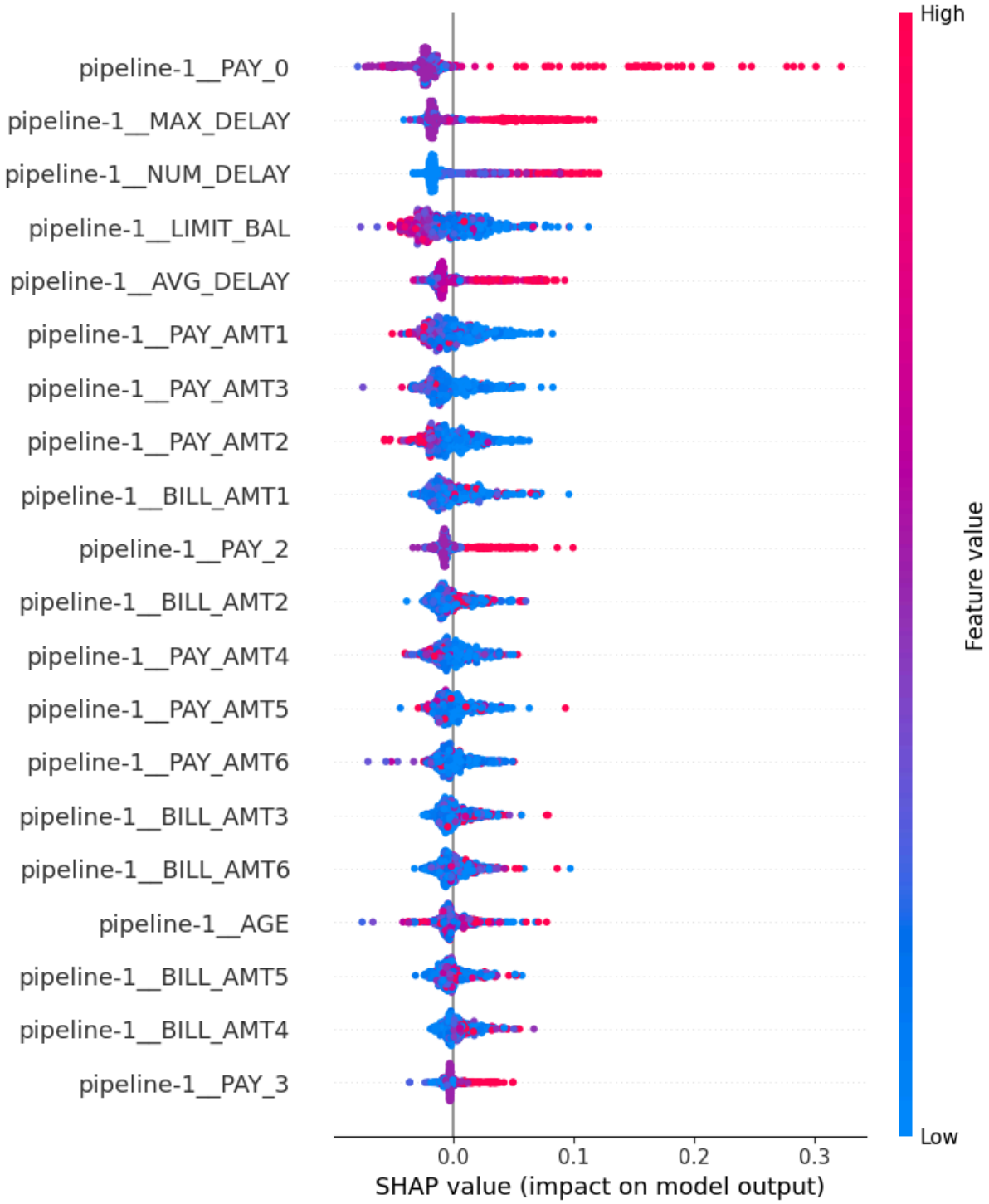
In [145…

```
# for random forest
# taken from lecture 12, CPSC 330 2025W2

import shap

feature_names = preprocessor.get_feature_names_out()

X_train_enc = pd.DataFrame(
    data=preprocessor.transform(X_train),
    columns=feature_names,
    index=X_train.index,
)

X_test_enc = pd.DataFrame(
    data=preprocessor.transform(X_test),
    columns=feature_names,
    index=X_test.index,
)
model = pipe_rf.named_steps["randomforestclassifier"]
model.fit(X_train_enc, y_train)

explainer = shap.TreeExplainer(model)

X_sample = X_train_enc[0:500]  # getting the shap values of every row in the train
```

```
train_shap_values = explainer(X_sample)
shap.initjs()
```

In [146…

```
shap.summary_plot(train_shap_values[:,:,1], X_sample)
```



From just the first 500 samples, we can see that PAY_0 is the most important feature for our random forest model. From this graph, we see that having a high PAY_0 value is heavily correlated with a higher SHAP value, and therefore a higher chance to default. We can also

see that, in general, having a greater delay on payments is positively correlated with SHAP value as well but to a lesser degree than PAY_0.

We can see that the 4 most impactful features are PAY_0, and the 3 features describing payment delays. All 4 of these features are positively correlated with SHAP value, indicating that having higher delays on payments greaty increases a clients chance to default.

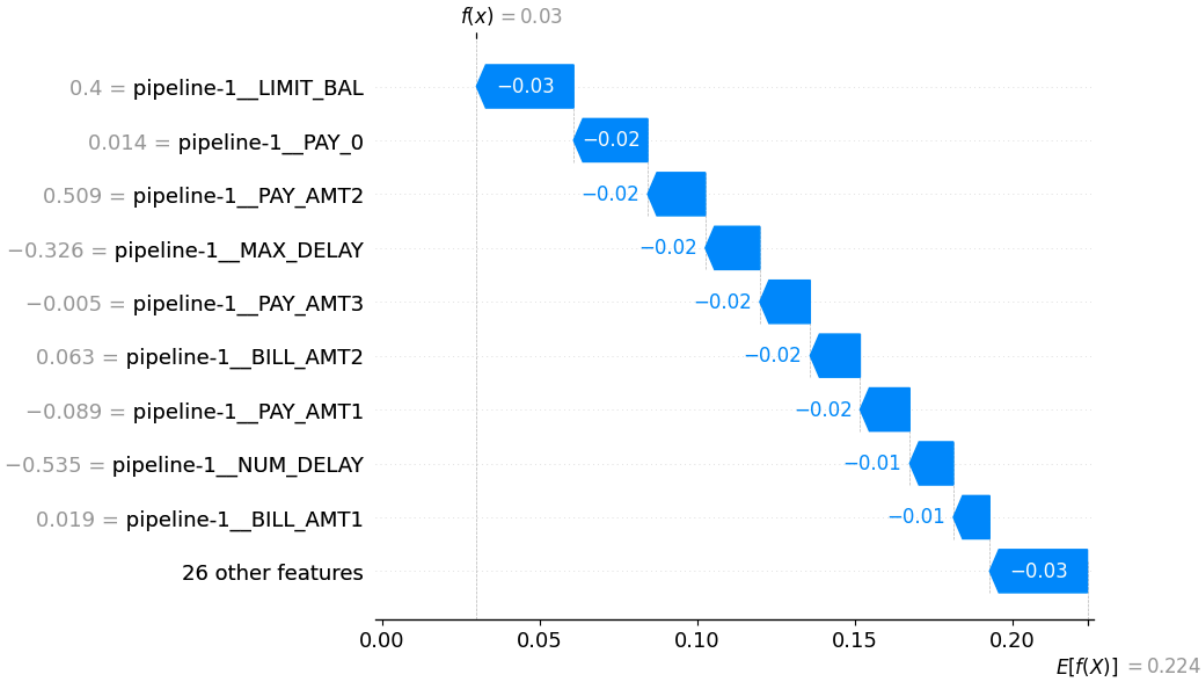
Some features that are inversely porportional to SHAP value are balance limit and payment amounts. This indicates that having a higher balance limit and higher payment amounts will generally decreases a client's chance of defaulting.

To reinforce these findings, we will look specifically at 2 samples and their waterfall plots.

In [147…

```
idx = 389
print(idx)
print (X_sample.iloc[idx])
shap.plots.waterfall(train_shap_values[:,:,1][idx])
```

```
389
pipeline-1__LIMIT_BAL        0.400304
pipeline-1__AGE              1.790972
pipeline-1__PAY_0            0.013770
pipeline-1__PAY_2            0.114774
pipeline-1__PAY_3            0.143483
pipeline-1__PAY_4            0.192754
pipeline-1__PAY_5            0.232531
pipeline-1__PAY_6            0.257059
pipeline-1__BILL_AMT1        0.019450
pipeline-1__BILL_AMT2        0.062987
pipeline-1__BILL_AMT3       -0.051401
pipeline-1__BILL_AMT4       -0.112036
pipeline-1__BILL_AMT5       -0.349322
pipeline-1__BILL_AMT6       -0.598614
pipeline-1__PAY_AMT1        -0.089391
pipeline-1__PAY_AMT2         0.509014
pipeline-1__PAY_AMT3        -0.004703
pipeline-1__PAY_AMT4        -0.179737
pipeline-1__PAY_AMT5        -0.246381
pipeline-1__PAY_AMT6         4.293467
pipeline-1__BILL_PAY1       -0.033706
pipeline-1__BILL_PAY2       -0.041886
pipeline-1__BILL_PAY3       -0.043319
pipeline-1__BILL_PAY4       -0.050018
pipeline-1__BILL_PAY5       -0.051039
pipeline-1__BILL_PAY6       -0.052235
pipeline-1__MAX_DELAY       -0.325570
pipeline-1__NUM_DELAY       -0.534714
pipeline-1__AVG_DELAY        0.188425
ordinalencoder__EDUCATION    2.000000
onehotencoder__SEX_2         1.000000
pipeline-2__MARRIAGE_0       0.000000
pipeline-2__MARRIAGE_1       1.000000
pipeline-2__MARRIAGE_2       0.000000
pipeline-2__MARRIAGE_3       0.000000
Name: 13930, dtype: float64
```

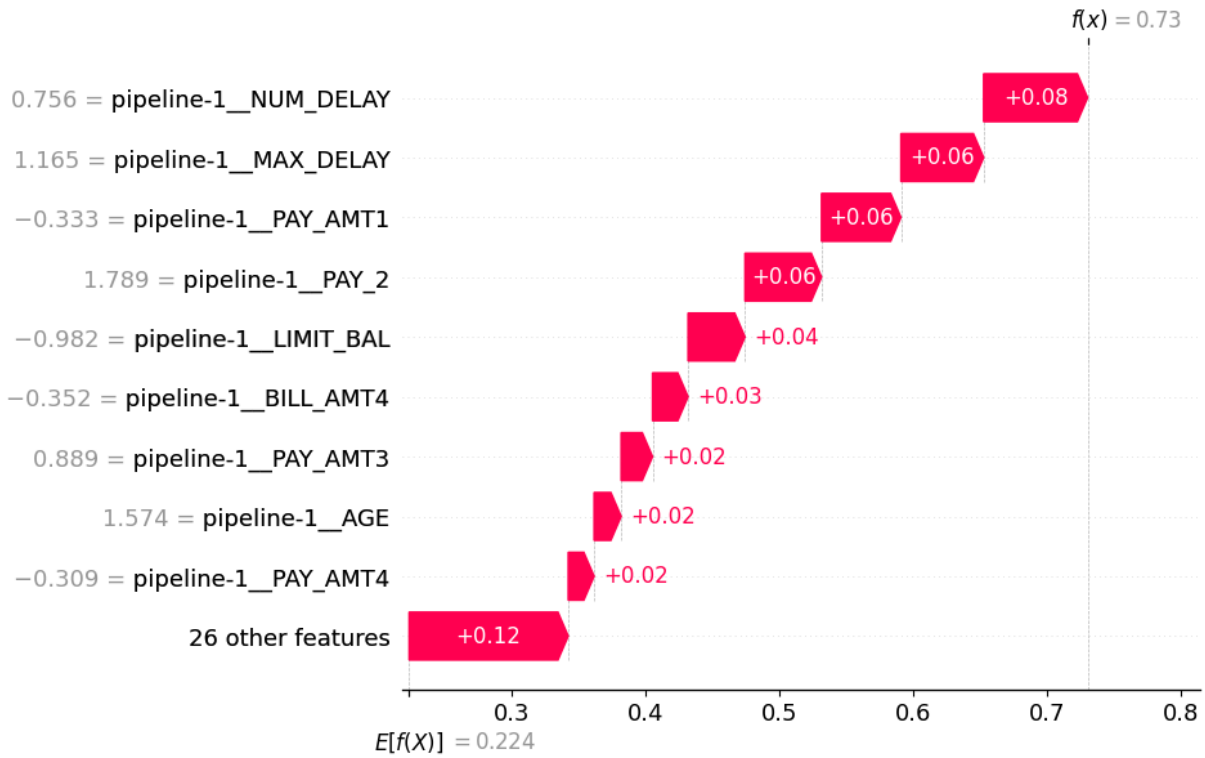


This sample shows a client who has low PAY values and low delays. Additionally, this client had a relatively high PAY_AMT2 and LIMIT_BAL. The affirms our previous analysis on the features.

In [148…

```
idx = 356
print(idx)
print (X_sample.iloc[idx])
shap.plots.waterfall(train_shap_values[:,:,1][idx])
```

```
356
pipeline-1__LIMIT_BAL       -0.982189
pipeline-1__AGE              1.573874
pipeline-1__PAY_0            0.906278
pipeline-1__PAY_2            1.789147
pipeline-1__PAY_3            0.143483
pipeline-1__PAY_4           -0.663024
pipeline-1__PAY_5           -0.646840
pipeline-1__PAY_6           -1.485154
pipeline-1__BILL_AMT1       -0.207354
pipeline-1__BILL_AMT2       -0.203112
pipeline-1__BILL_AMT3       -0.148527
pipeline-1__BILL_AMT4       -0.352190
pipeline-1__BILL_AMT5       -0.660751
pipeline-1__BILL_AMT6       -0.650606
pipeline-1__PAY_AMT1        -0.333097
pipeline-1__PAY_AMT2        -0.129216
pipeline-1__PAY_AMT3         0.889050
pipeline-1__PAY_AMT4        -0.309323
pipeline-1__PAY_AMT5        -0.312050
pipeline-1__PAY_AMT6        -0.289117
pipeline-1__BILL_PAY1       -0.033706
pipeline-1__BILL_PAY2       -0.041886
pipeline-1__BILL_PAY3       -0.043319
pipeline-1__BILL_PAY4       -0.050018
pipeline-1__BILL_PAY5       -0.051039
pipeline-1__BILL_PAY6       -0.052235
pipeline-1__MAX_DELAY        1.164940
pipeline-1__NUM_DELAY        0.755826
pipeline-1__AVG_DELAY        0.018505
ordinalencoder__EDUCATION    1.000000
onehotencoder__SEX_2         0.000000
pipeline-2__MARRIAGE_0       0.000000
pipeline-2__MARRIAGE_1       1.000000
pipeline-2__MARRIAGE_2       0.000000
pipeline-2__MARRIAGE_3       0.000000
Name: 27578, dtype: float64
```



This sample shows a client who has relatively high delays on payments and high PAY values. As a result, the client was predicted to default. This also affirms our previous analysis.

12. Results on the test set

---

rubric={points:10}

Your tasks:

1. **Evaluate on the test set**: Apply your best-performing model to the test data and report the test scores.
2. **Compare and reflect**: Compare the **test scores** with the **validation scores** from previous experiments. Discuss the consistency between them. How much do you **trust your results**? Reflect on whether you might have encountered **optimization bias**.
3. **Explain individual predictions**: Select one or two examples from your test predictions and use an interpretation method (e.g., **SHAP force plots**) to explain these individual predictions.

Solution_12

Points: 10

Despite having the lowest test accuracy (75%), our best performing model in terms of f1 and recall was our logistic regression model, with an f1 score of 53% and a recall of 63%.

In [ ]:
```
pipe_lr = make_pipeline(preprocessor, LogisticRegression(random_state=123, C=0.01,
pipe_lr.fit(X_train, y_train)
y_pred = pipe_lr.predict(X_test)

score_lr = pipe_lr.score(X_test, y_test)
precision_lr = precision_score(y_test, y_pred)
recall_lr = recall_score(y_test, y_pred)
f1_lr = f1_score(y_test, y_pred)

print(f"Accuracy: {score_lr:.3f}")
print(f"Precision: {precision_lr:.3f}")
print(f"Recall: {recall_lr:.3f}")
print(f"F1 Score: {f1_lr:.3f}")
```

```
Accuracy: 0.741
Precision: 0.431
Recall: 0.614
F1 Score: 0.506
```

Compared to the validation scores from the training data, all of the values are actually very similar, just a little bit lower which is to be expected. Test precision is 43% instead of 45%, test accuracy is 74% instead of 75%, test recall is 61% instead of 63%, and f1 is 51% instead of 53%. The test and training scores are likely so similar due to the model balancing the data.

I would generally trust these results, as there doesn't seem to be any overfitting (difference between test and train accuracy is small) and this model is performing extremely similarly on unseen data as it did on training data, meaning there likely isn't much risk of optimization bias.

In [127…
```
import shap

X_test_shap = pipe_lr.named_steps["columntransformer"].transform(X_test)
explainer_lr = shap.LinearExplainer(pipe_lr.named_steps["logisticregression"], X_te
shap_values_lr = explainer_lr.shap_values(X_test_shap)
shap.initjs()

feature_names = pipe_lr.named_steps["columntransformer"].get_feature_names_out()
X_test_shap = pd.DataFrame(X_test_shap, columns=feature_names, index=X_test.index)
```

In [128…
```
shap.force_plot(
    explainer_lr.expected_value,
    shap_values_lr[0, :],
```

```
    X_test_shap.iloc[0, :],
)
```

Out[128…

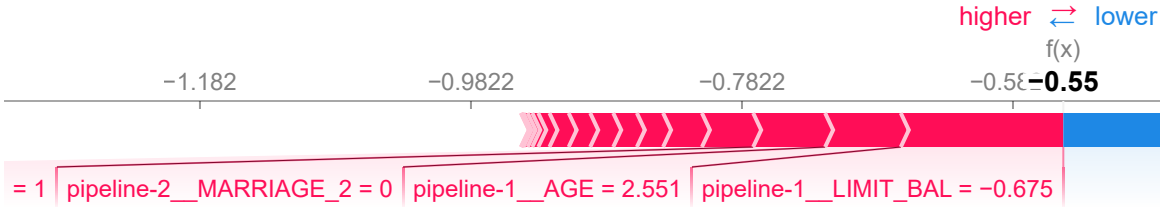
Example 1:

The value of this plot is way below zero, meaning this person is lower risk of defaulting payments. NUM_DELAY and BILL_PAY_4 were the biggest positive coefficients and LIMIT_BAL was the biggest negative coefficient. This makes sense as someone with a shorter delay of repayment is likely to continue repaying their balances on time, while having a worse credit limit (LIMIT_BAL) might make someone more at risk of defaulting a payment.

In [129…

```
shap.force_plot(
    explainer_lr.expected_value,
    shap_values_lr[1, :],
    X_test_shap.iloc[1, :],
)
```

Out[129…



Example 2:

This person is predicted by the model to be slightly more at risk of defaulting a payment, although they are still below zero, meaning they are likely safe and will not default. For this person, their main positives are NUM_DELAY and MAX_DELAY while their main negatives are their AGE and LIMIT_BAL. This one is a bit more ambiguous for the model because the model is comparing their demographic (AGE) to their payment behaviours (NUM_DELAY, MAX_DELAY). However, the model ends up predicting that this person will not default their payments despite their AGE being flagged as they have shown a history of credit repayment, leading to their overall negative score.

13. Summary of results

rubric={points:12}
Imagine you are preparing to present the summary of your results to your boss and co-workers.

Your tasks:

1. **Summarize key results**: Create a clear and concise table highlighting your most important results (e.g., models compared, validation/test scores, key observations).
2. **Write concluding remarks**: Summarize your main takeaways from the project, including what worked well and what did not.
3. **Propose future improvements**: Discuss ideas or approaches you did not try but that could potentially improve **performance** or **interpretability**.
4. **Report final results**: Report your **final test score** and the **metric** you used.

Solution_13

Points: 12

Model	Accuracy	F1 Score	Precision	Recall	Observations
Decission Tree Classifier	0.818095	0.446214	0.696874	0.329344	The decision tree classifier has an accuracy around 9% more than the dummy classifier while having relatively low F1, precision, and recall scores. From our testing, this model has a high tendency to overfit.

Model	Accuracy	F1 Score	Precision	Recall	Observations
Linear Regression	0.751286	0.530409	0.458571	0.629049	Even though this model had generally a lower accuracy, it outperformed the others when it came to F1 and recall scores, making it the best model for our specific application. This model was also the only model to have a hyperparameter which directly helped balance the default to non-default imbalance within the dataset. The behaviour of this model was also very erratic during hyperparameter testing, but ultimately produced results similar to the validation scores when given test data.
K nearest neighbors	0.814286	0.450158	0.664440	0.340660	This model performed very similarly to the decision tree, but overall had the worst performance out of the three models. This was surprising as in the associated research paper KNN was one of the models with the least error. It's possible that this model was just poorly suited for the way we preprocessed our data. This model didn't really overfit or underfit.
Random Forest Classifier	0.816048	0.480569	0.650813	0.381186	Like the decision tree model, this model has a high tendency to overfit in spite of hyperparameter optimization. Even though this model performed the best in terms of accuracy, we decided to use F1 score, precision, and recall as our main metrics. Additionally, the random nature of this model caused slight differences in scores each time the model was run.

Summary:

When it came to optimizing the hyperparameters for our random forest model, we found out pretty quickly that performing grid search with many large values for max_depth and n_estimators resulted in extremely large computational costs. This limited us to use smaller ranges and smaller values. Performing a different method of hyperparameter optimization might of allowed us to optimize our model further, allowing for better scores. Similarly, for feature selection, RFE took way too much time to be efficiently used inside our pipeline. It's likely that with different models, as random forest is very computationally expensive, RFE would have improved overall model accuracy without needing as much time.

Given such a large dataset, calculating every single SHAP score was also very computationally expensive, and our notebooks crashed multiple times adjusting the amount of samples we could read. Because of this, we could only read 500 samples, which doesn't give us the full SHAP score on our model's predictions.

Future Improvements:

We had a couple ideas we wanted to try but weren't able to for the sake of time. One thing that could be improved was our choice of ensemble model, as random forest performed pretty poorly. We thought of using a different model, such as xgboost or a gradient boost model, but ultimately went with random forest beause we were curious how it would perform compared to a single decision tree model (it ended up performing pretty badly unfortunately).

We also thought about excluding gender as a feature, as the relationship between gender and likelihood of defaulting was very similar for both cases, but ended up keeping it because we were worried it would have a strong relationship with another factor. However, after doing feature selection, where both decision tree and random forest elected to eliminate the male gender as a feature (that being the only feature removed by random forest), it's possible that removing gender may have boosted the performance of the model.

Final Results:

Our final test score using logistic regression with hyperparameters C=0.01, max_iter=100, and class_weight="balanced", was 0.741 and we primarily used our f1 and recall scores to judge the performance of this model against the other models.

Accuracy: 0.741, Precision: 0.431, Recall: 0.614, F1 Score: 0.506

14. Your takeaway

rubric={points:2}

Your tasks:

What is your biggest takeaway from the supervised machine learning material we have learned so far? Please write thoughtful answers.

Solution_14

Points: 2

Our biggest takeaway would probably be about how complex even the most basic machine learning models can be. Right now we're only given the more basic pre-made models from sklearn, but even so they're able to detect so many intricate patterns based off of seemingly unconnected data. It's even more amazing how something as simple as adjusting a single hyperparameter can lead to such drastic changes in model behaviour. It's just very cool being able to observe machines detecting patterns which would otherwise be impossible for humans to connect. We imagine that in future courses (like 340) that the level of pattern detection and complexity will be even more incredible as we actually begin building our own models. It's also been interesting seeing the different ways machines can fail when given incorrect hyperparameters, and how they can follow the wrong trail of patterns.

Another key takeaway would be how much time we spent on data processing vs model creation. We found that most of the time spent on this project was towards cleaning the data and thinking about ways we could add to or change the data to potentially boost the performance of our later models. Actually creating the models took way less time than the first parts of this assignment, which came as a suprise to us.

Before submitting your assignment, please ensure you have followed all the steps in the **Instructions** section at the top.

Submission checklist

- ☐ Restart the kernel and run all cells (▶️ ▶️ button)
- ☐ Make sure to push the most up to date version of your homework assignment to your GitHub repository so that we can use it for grading if there are any problems with your submission on Gradescope.
- ☐ The `.ipynb` file runs without errors and shows all outputs.
- ☐ Only the `.ipynb` file and required output files are uploaded (no extra files).
- ☐ If the `.ipynb` file is too large to render on Gradescope, upload a Web PDF and/or HTML version as well.

This was a tricky one but you did it 👏 👏!